

MyHealer

A Smart Hospital Management System with AI-Based X-ray Fracture Detection

By

Lee Wei Hang



FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
PENANG

ACADEMIC YEAR
2026/27

MyHealer

A Smart Hospital Management System with AI-Based X-ray Fracture Detection

By

Lee Wei Hang

Supervisor: Ms. Kee Sim Ee

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Bachelor in Software Engineering (Honours)

Department of Software Engineering and Technology
Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Penang

Copyright by Tunku Abdul Rahman University of Management and Technology.

All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

Hang_____

Lee Wei Hang

Bachelor in Software Engineering (Honours)

ID: 25PMR11598

Acknowledgement

I would like to express my sincere gratitude to everyone who has contributed to the completion of this final year project, MyHealer: Integrated Hospital Management System with AI-Assisted Bone Fracture Detection. Their guidance, support, encouragement, and advice have been valuable throughout the development of this project.

First and foremost, I would like to extend my deepest appreciation to my project supervisor, Ms. Kee Sim Ee, for her continuous guidance, constructive feedback, and valuable suggestions throughout the project. Her advice has been important in shaping the direction of the project, especially in refining the project scope, strengthening the focus on AI-assisted bone fracture classification, and ensuring that the system remains feasible within the final year project timeline. I sincerely appreciate her patience, support, and professional guidance during the completion of this work.

I would also like to thank the lecturers and academic staff of the Faculty of Computing and Information Technology, Tunku Abdul Rahman University of Management and Technology, Penang Campus, for the knowledge, skills, and learning opportunities provided throughout my studies. The technical and theoretical knowledge gained from the programme has greatly supported the planning, analysis, design, and development of this project.

In addition, I would like to express my appreciation to my family and friends for their constant encouragement, understanding, and moral support. Their belief in my ability has motivated me to continue working through the challenges faced during the project. Their support has helped me remain focused and determined throughout this final year project journey.

Finally, I would like to thank everyone who has contributed directly or indirectly to this project, including those who provided suggestions, feedback, technical assistance, and encouragement during the development process. Every contribution, whether large or small, has been meaningful in helping me complete this project successfully. I am truly grateful for the support and guidance received throughout the completion of this final year project.

Table of Contents

2 Literature Review	2
2.1 Literature Review	2
2.1.1 Introduction	3
2.1.2 Comparison of Existing Systems	5
2.1.3 Mobile Application Framework	9
2.1.4 Web Front-End Framework	13
2.1.5 Back-end Framework	17
2.1.6 Database	23
2.1.7 Application Programming Interface Architectural Style	28
2.1.8 Deep Learning for Bone Fracture Classification	33
2.1.8.1 Convolutional Neural Network	35
2.1.8.2 CNN Architectures for Image Classification	39
2.1.8.3 Existing Studies on Automated Bone Fracture Detection and Classification	44
2.1.8.4 Dataset for Bone Fracture Classification	49
2.1.8.5 Grad-CAM for Model Interpretability	54
2.1.8.6 AI Framework for Model Training	56
2.1.8.7 AI Model Serving Framework	60
2.2 Feasibility Study	65
2.2.1 Technical Feasibility	66
2.2.2 Economic Feasibility	68
2.2.2.1 Cost and Benefit Summary	69
2.2.2.2 Tangible Cost	72
2.2.2.3 Operating and Maintenance Cost	74
2.2.2.4 Intangible Cost	76
2.2.2.5 Economic Feasibility Conclusion	77
2.2.3 Operational Feasibility	78
2.2.4 Schedule Feasibility	80
2.2.5 Legal Feasibility	82
2.2.6 Ethical Feasibility	84
2.3 Chapter Summary and Evaluation	86
References	89

Chapter 2

Literature Review

2 Literature Review

2.1 Literature Review

This chapter presents a comprehensive literature review supporting the development of MyHealer, an AI-assisted bone fracture classification system embedded within an integrated hospital management platform. Its primary objective is to critically examine existing diagnostic and hospital systems, review the state of the art in deep learning for medical image analysis, and evaluate the technologies available for implementation, thereby establishing an evidence-based foundation for the design and development phases that follow.

The chapter begins by examining the diagnostic problem that motivates MyHealer: the workload placed on doctors and radiologists interpreting X-ray images, and the resulting risk of delay and error. It then reviews existing hospital and healthcare applications to determine the extent to which automated diagnostic support is currently available, establishing the research gap that MyHealer addresses. The chapter subsequently undertakes an in-depth review of deep learning for bone fracture classification, covering convolutional neural networks and their architectures, existing studies on automated fracture detection and classification, the dataset selected for model training, the Grad-CAM technique used to visualise model predictions, and the frameworks used to train and serve the model. As the hospital management functions of MyHealer, including patient queue management, are supporting features rather than the project's core contribution, the chapter also briefly reviews the technologies enabling these functions before evaluating the mobile, web, back-end, database, and API technologies used to build the wider system.

By undertaking this review, the chapters that follow are grounded in an informed understanding of existing diagnostic systems, deep learning techniques, and implementation technologies. This foundation supports the methodology, requirements analysis, and system design presented in the subsequent chapters, and ensures that MyHealer's AI-assisted fracture classification module is developed on a solid evidential basis.

2.1.1 Introduction

MyHealer is an AI-assisted diagnostic support system developed to address one of the most consequential bottlenecks in hospital radiology: the interpretation of X-ray images to detect bone fractures. At its core, MyHealer employs a convolutional neural network (CNN), trained from scratch on a large collection of labelled X-ray images, to classify a radiograph as fractured or non-fractured. Rather than functioning as a black box, the system uses Gradient-weighted Class Activation Mapping (Grad-CAM), a technique that produces a visual heatmap highlighting the image regions most responsible for a model's prediction (Selvaraju et al., 2017), together with a confidence score expressing how certain the model is of its classification. In this way, MyHealer does not simply replace a doctor's judgement; it supports it, drawing attention to the areas of an X-ray most likely to contain a fracture and allowing the attending doctor to verify, and if necessary override, the system's assessment.

Beyond this diagnostic core, MyHealer is embedded within a broader hospital management ecosystem. Patients access a mobile application to book appointments, monitor their position in a smart queue, and view their medical records, while hospital staff, doctors, pharmacists, and administrators interact with the system through a web-based platform covering consultation, prescription, pharmacy, and hospitalisation workflows. Notably, the smart queue management feature, although a visible convenience to patients, is not an independent solution to hospital inefficiency; rather, it is a downstream benefit made possible by the diagnostic core. When a doctor is aided by an AI classification rather than relying solely on manual interpretation, fractures can be identified more accurately and with fewer misses (Duron et al., 2021), reducing the diagnostic errors and repeat presentations that contribute to congestion in the patient queue. MyHealer's hospital management functions are therefore best understood as the delivery vehicle for its AI-assisted diagnosis, rather than as a parallel objective of equal weight.

The significance of this AI-first approach is grounded in the scale of the diagnostic problem it addresses. Reading X-rays accurately under time pressure is a persistent challenge in clinical practice: even in well-resourced emergency departments, studies have found that a measurable proportion of fractures are missed on initial radiographic interpretation, with one systematic analysis of extremity fractures reporting an overall miss rate of 3.7% (Wei et al. , 2006), and the risk rising further for clinicians with less specialist experience. This risk is compounded in a healthcare system such as Malaysia's, where the public sector faces a deficit

of nearly 11,000 specialist doctors, requiring individual physicians to manage between 80 and 100 patients per day (Boo Su-Lyn, 2025), leaving little time for the careful, unhurried image review that reduces diagnostic error. At the same time, increasing demand on radiologists, particularly in low- and middle-income countries, has been identified as a key driver of the growing need for diagnostic automation, with deep learning consistently proposed as a technology capable of meeting this need (Aggarwal et al., 2021). It is this convergence, rising diagnostic workload on one hand and increasingly capable, accessible deep learning tools on the other, that motivates the development of MyHealer's AI-assisted fracture classification module as the central contribution of this project.

2.1.2 Comparison of Existing Systems

Existing solutions relevant to MyHealer fall into two distinct groups. The first comprises integrated hospital and patient-management applications, that is, systems that digitalise appointments, records, and related administrative workflows. The second comprises standalone artificial-intelligence tools that analyse radiographs to detect fractures. What distinguishes MyHealer is that it unites these two capabilities, embedding AI-assisted fracture classification within an integrated, freely accessible hospital-management system. To establish this gap, this section first compares MyHealer against three representative integrated systems, namely MySejahtera (Malaysia's national public-health application), KPJ Cares (a major private-hospital application), and Epic MyChart (a globally established patient portal), taking the presence of AI-based diagnostic support as the defining point of comparison. It then considers the standalone AI fracture-detection tools that occupy the second group. Table 2.1 summarises the comparison of the integrated systems.

Table 2.1: Comparison of Existing Systems

Capability	MySejahtera (Ministry of Health Malaysia, n.d.)	KPJ Cares (KPJ Healthcare Berhad, n.d.)	Epic MyChart (Epic Systems Corporation, n.d.)	MyHealer
System type	Public health app (MOH)	Private hospital app (single network)	Enterprise EHR patient portal	Integrated smart hospital with AI diagnosis
AI-based fracture diagnosis	No	No	No (AI limited to scheduling/billing)	Yes (CNN + Grad-CAM)
Diagnostic decision support for doctors	No	No	No	Yes

Appointment booking	Yes (government facilities)	Yes (KPJ network)	Yes	Yes
Real-time queue monitoring	No	No	No	Yes
Medical records access	Yes (limited)	Partial	Yes (comprehensive)	Yes
Prescription / e-pharmacy	No	Yes	Yes	Yes
Hospitalisation management (admission, ward/bed, discharge, billing)	No	No	Partial (inpatient information only)	Yes
Integrated staff & admin platform	No (patient-facing only)	No (patient-facing only)	Yes (via Epic EHR)	Yes
Cost & accessibility	Free (public)	Free app (private care chargeable)	Very high (enterprise licensing)	Free / open-source prototype
Suitability for a Malaysian public-hospital context with AI support	Low	Low–Medium	Low (cost barrier)	High

The most significant finding in Table 2.1 concerns diagnostic intelligence. None of the three integrated systems provides any form of AI-based fracture diagnosis or medical-image analysis. MySejahtera and KPJ Cares are oriented toward public-health administration and private-hospital engagement respectively (Ministry of Health Malaysia, n.d.; KPJ Healthcare Berhad, n.d.), and neither analyses radiographs at all. Epic MyChart, although the most sophisticated of the three and increasingly incorporating AI, applies that intelligence to administrative tasks such as scheduling and billing rather than to diagnosis, and does not interpret medical images (Epic Systems Corporation, n.d.). In every case, the interpretation of an X-ray remains an entirely manual task. This absence of diagnostic decision support is precisely the gap that MyHealer's AI fracture-classification module is designed to fill.

The capability that these integrated systems lack does, however, exist as a separate class of product. Commercial, regulator-cleared AI fracture-detection tools are now deployed in clinical radiology. The most established, Gleamer's BoneView, was the first such solution to receive FDA clearance, and it analyses radiographs to detect and localise fractures before returning its findings to a radiologist for final confirmation (Applied Radiology, 2022). The clinical value of this class of tool is supported by peer-reviewed evidence: in a multicentre study, an AI aid improved the sensitivity and specificity of both radiologists and emergency physicians in detecting appendicular fractures, without increasing reading time (Duron et al., 2021). Comparable tools, such as Radiobotics' RBfracture, have since entered the market. Critically, however, these are proprietary, enterprise-grade tools that operate within the radiology back-end of well-resourced institutions. They are neither embedded in patient-facing hospital-management platforms nor readily adopted in a resource-constrained public sector such as Malaysia's, where digital adoption remains limited (Naim et al., 2025). They therefore occupy the opposite position to the integrated applications, offering powerful diagnosis with no integration or accessibility, rather than integration with no diagnosis.

Setting the diagnostic dimension aside, the integrated systems differ across the supporting features that MyHealer also provides. All three offer appointment booking, but none provides real-time queue monitoring, a feature MyHealer includes as a secondary benefit of streamlining the diagnostic workflow. Record access ranges from limited (MySejahtera) to comprehensive (MyChart); prescription and e-pharmacy support is present in KPJ Cares and MyChart but absent in MySejahtera; and full hospitalisation management, covering admission, ward and bed allocation, discharge, and billing, is largely absent across all three.

MyHealer matches or extends these functions while adding the diagnostic capability none of them possess.

Cost and accessibility further separate the options. MySejahtera and KPJ Cares are free to the user but confined to their respective public and single-network contexts, whereas Epic MyChart, as the patient portal of an enterprise electronic-health-record system, carries the licensing and implementation costs characteristic of such enterprise deployments. MyHealer, built entirely on free and open-source technologies, is positioned to deliver both integrated management and AI diagnosis without such barriers.

On this basis, the suitability of each system for the specific goal of AI-assisted diagnostic support within an accessible hospital platform can be assessed. The integrated applications rate low: they are competent management tools but offer no diagnostic intelligence. The standalone AI tools, though diagnostically capable, are unsuitable in this context owing to their proprietary, enterprise-oriented, and non-integrated nature. MyHealer alone occupies the intersection of the two groups, combining integrated hospital management with accessible AI-assisted fracture diagnosis, which constitutes the central gap this project addresses.

2.1.3 Mobile Application Framework

The patient-facing component of MyHealer is delivered as a mobile application, which serves as the primary point of contact between patients and the hospital. Through this application, patients book and manage appointments, monitor their queue position in real time, view their medical records and diagnostic results, and receive notifications. Because the application is the channel through which patients experience the system, the framework used to build it is an important design decision. It must produce a responsive and reliable interface, reach patients regardless of the device they own, and remain feasible to build and maintain by a single developer within the project timeline.

Two requirements are especially significant. The first is broad device reach. Patients cannot be assumed to use any single operating system, so the application must run on both Android and iOS. Given that smartphone penetration in Malaysia exceeds 90% (Statista, 2025), a mobile-first channel is well suited to the target users, but only if it is available across both major platforms. The second requirement is timely communication. Much of the application's value, such as queue-position updates and appointment reminders, depends on the ability to push notifications directly to the patient's device, so the chosen framework must support reliable integration with push-notification services. In addition, because MyHealer is developed by a single developer, maintaining two separate native codebases would roughly double the development and maintenance effort, which is difficult to justify for a project of this scope and strongly favours an approach that maximises code reuse across platforms.

Mobile applications can be built using one of three broad approaches: native development, cross-platform development, and progressive web applications. Table 2.2 compares these categories.

Table 2.2: A Summarized Comparison of Mobile Application Framework Categories

	Native	Cross Platform	Progressive Web App
Examples	Kotlin (Android), Swift (iOS)	Flutter, React Native	Web apps (React, Vue) as installable web
Strengths	Best performance; full device and API access; native look and feel	Single codebase for iOS and Android; faster development and lower cost; large developer communities	No installation required; instant updates; wide reach through the browser
Weaknesses	Separate codebase per platform; higher development and maintenance cost; longer time to market	Possible performance overhead relative to native; access to some platform-specific features can be limited	Limited device-feature and offline access; dependent on the browser; less reliable notifications

As shown in Table 2.2, each category offers a different balance of performance, reach, and development effort. Native development, using platform-specific languages and tools, delivers the best performance and the fullest access to device features, but it requires a separate codebase for each platform, increasing development and maintenance cost (Biørn-Hansen, Grønli, & Ghinea, 2018). Progressive web applications avoid installation and are easy to update, but they offer more limited access to device features and offline capability and depend on the browser (Biørn-Hansen et al., 2018). Cross-platform frameworks occupy a middle position, allowing a single codebase to target both major platforms and thereby substantially reducing development effort and time to deployment, which has driven their strong adoption in industry (Biørn-Hansen, Grønli, Ghinea, & Alounch, 2019). Their principal trade-off is that access to certain platform-specific features can be more limited than in native development (Biørn-Hansen et al., 2019).

For MyHealer, the cross-platform approach is the most appropriate. The application does not depend on advanced, platform-specific hardware features that would demand native development, since its core functions, namely appointment booking, queue monitoring,

record viewing, and notifications, are well within the capabilities of modern cross-platform frameworks. At the same time, the single-codebase model directly addresses the solo-developer constraint and the requirement to reach both Android and iOS users. Progressive web applications are less suitable, because the notification and device-integration requirements of the patient application are better served by an installed application. Having selected the cross-platform approach, the two leading frameworks in this category, Flutter and React Native, are compared in Table 2.3.

Table 2.3: A Summarized Comparison of Cross-Platform Mobile Application Frameworks

Aspect	Flutter	React Native
Language	Dart	JavaScript / TypeScript
Rendering approach	Own graphics engine, does not rely on native UI components	Native UI components accessed through a bridge
Performance	High, close to native	Good, near-native
Development	Hot reload; comprehensive built-in widget set	Hot reload
Ecosystem and community	Growing rapidly; younger than the JavaScript ecosystem	Mature, backed by the large JavaScript ecosystem
Suitability for MyHealer	High	Medium to High

Flutter was selected for MyHealer's mobile application. Developed by Google, Flutter uses the Dart language and compiles ahead of time to native code, rendering the interface through its own graphics engine rather than relying on platform UI components (Google, n.d.-a). This design gives it consistent, high-performance rendering across Android and iOS from a single codebase, which suits a patient-facing application that must feel responsive and behave identically on both platforms. Flutter's hot-reload capability, which reflects code changes in the running application almost immediately, accelerates development and iteration (Google, n.d.-a), an advantage for a solo developer working to a fixed timeline. It is free and open source, and its documentation and widget library are comprehensive.

React Native, developed by Meta, is a strong alternative. It uses JavaScript and renders through native UI components (Meta, n.d.-b), and it benefits from the large JavaScript ecosystem and a long track record in industry. The considerations that weigh against it for this project are relative rather than absolute: Flutter's self-contained rendering engine produces more consistent cross-platform interface behaviour, which reduces the platform-specific adjustments that can arise from React Native's reliance on native components.

Flutter is not without limitations. It relies on Dart, a language less widely adopted than the JavaScript used by React Native, so fewer developers are already familiar with it, and its package ecosystem, although growing rapidly, is younger than the JavaScript ecosystem (Biørn-Hansen et al., 2019). For MyHealer these limitations carry little weight, because Dart is straightforward to learn, the application is being built from scratch rather than adapted from existing JavaScript code, and its feature set is fully covered by Flutter's standard widgets and available packages. On balance, Flutter's consistent high-performance rendering, rapid development workflow, and single-codebase reach make it the most suitable choice for the MyHealer patient application.

2.1.4 Web Front-End Framework

The web-based component of MyHealer is used by doctors, hospital staff, pharmacists, and administrators to access the system functions that are not suitable for the patient-facing mobile application. Doctors require a web dashboard to review patient information, record consultation outcomes, upload X-ray images, view AI-assisted fracture classification results, inspect Grad-CAM heatmaps, and confirm diagnoses. Hospital staff require interfaces for registration, appointment handling, queue management, and radiology image management. Pharmacists require prescription and medication distribution functions, while administrators require dashboards for staff management, hospital operations, analytics, and reporting. Therefore, the selected front-end web framework must support responsive interfaces, reusable components, fast interaction, maintainable dashboard development, and effective integration with back-end APIs.

A front-end web framework is important because MyHealer involves several role-specific dashboards rather than a single static web page. Each role requires different screens, forms, tables, navigation flows, and data visualisations. A component-based front-end approach is therefore suitable because repeated interface elements, such as patient cards, prescription tables, appointment lists, alert panels, and diagnostic result views, can be developed as reusable components. This improves consistency and maintainability, especially for a solo developer working within the fixed FYP timeline. Modern front-end frameworks also support dynamic user interfaces, enabling the page to update when new data is received from the back-end API without requiring a full page reload. This is important for MyHealer because staff dashboards must display updated appointments, queues, prescription statuses, and diagnostic records efficiently.

Three commonly used front-end web technologies were considered for MyHealer: Angular, React, and Vue.js. Angular is a complete web framework maintained by Google and is designed to support scalable applications through a broad set of tools, APIs, and libraries (Google, n.d.-b). React is a JavaScript library for building web and native user interfaces using reusable components (Meta, n.d.-a). Vue.js is a progressive JavaScript framework designed to be flexible and incrementally adoptable for different scales of web applications (Vue.js, n.d.). Table 2.4 summarises the comparison of these three options.

Table 2.4: A Summarized Comparison of Front-end Web Frameworks

Aspect	Angular (Google, n.d.-b)	React (Meta, n.d.-a)	Vue.js (Vue.js, n.d.)
Type	Full front-end framework	UI library	Progressive front-end framework
Main language	TypeScript	JavaScript / TypeScript	JavaScript / TypeScript
Development approach	Structured and opinionated framework with built-in tools	Component-based library with flexible ecosystem	Progressive framework that can be adopted gradually
Strengths	Strong structure; suitable for large enterprise systems; built-in routing, forms, and HTTP support	Reusable components; large ecosystem; flexible integration with different libraries; strong suitability for dashboards and single-page applications	Easier learning curve; flexible structure; suitable for small to medium applications
Weaknesses	Steeper learning curve; more complex setup; heavier structure for smaller projects	Requires additional libraries for routing, state management, and UI components; frequent ecosystem changes may require continuous learning	Smaller ecosystem than React; less commonly used in large enterprise settings
Suitability for MyHealer	Medium	High	Medium

Angular provides a highly structured development environment. Its official documentation describes Angular as a framework that provides tools, APIs, and libraries for building applications that scale with both team size and codebase size (Google, n.d.-b). This makes Angular suitable for large enterprise systems where strict architecture, built-in form handling, routing, dependency injection, and long-term structure are important. For a hospital

management system, these strengths are relevant because the system contains multiple user roles and workflows. However, Angular may be less suitable for MyHealer because it introduces a heavier framework structure and a steeper learning curve. Since MyHealer is developed by a single student developer, the additional complexity may increase development effort without providing a proportional benefit at the prototype stage.

Vue.js offers a simpler and more approachable alternative. It is described as a progressive framework that can be used in different ways depending on the scale and requirements of the application (Vue.js, n.d.). This flexibility is useful when a system needs to start small and expand gradually. Vue.js also provides a relatively easy learning curve, which makes it attractive for rapid development. However, MyHealer requires several interconnected dashboards, API-driven records, AI result screens, and administrative interfaces. Although Vue.js is technically capable of supporting these requirements, React provides a larger ecosystem and broader availability of reusable dashboard-oriented libraries, which makes React more suitable for a system that needs to be developed quickly with many interface components.

React was selected as the front-end web framework for MyHealer. React allows user interfaces to be built from reusable components, where smaller interface elements can be combined into complete screens and applications (Meta, n.d.-a). This directly matches the design needs of MyHealer because the doctor, staff, pharmacist, and administrator dashboards share repeated interface patterns such as tables, cards, forms, modals, status labels, and report panels. By using React components, these repeated elements can be created once and reused across different modules, improving interface consistency and reducing duplicated development effort. React is also well suited for single-page applications, where users interact with different views without full page refreshes. This is useful for MyHealer because the staff and doctor dashboards should provide a smooth workflow when switching between appointments, patient records, X-ray results, prescriptions, and reports.

Another reason React is suitable is its ecosystem. The 2025 Stack Overflow Developer Survey indicates that React remains one of the major web frameworks and technologies used by developers, reflecting its continued relevance and strong developer community (Stack Overflow, 2025). This is significant for MyHealer because a large ecosystem increases the availability of tutorials, documentation, reusable components, dashboard templates, and troubleshooting resources. In addition, React can be integrated with UI component libraries

such as Material UI, which provides ready-to-use React components based on Material Design (MUI, n.d.). Such libraries can reduce the time required to build professional-looking dashboards, forms, tables, buttons, and navigation components, which is valuable for a solo FYP developer.

React also aligns well with the overall architecture of MyHealer. The system uses a REST-based back-end API, so the web front end must communicate with the server to retrieve appointments, patient details, prescriptions, AI prediction results, and administrative reports. React applications commonly interact with APIs and update interface state dynamically, making it suitable for dashboard-based systems that display changing hospital data. This API-driven model also supports the separation of concerns between the front end and back end, where React focuses on the user interface while Django REST Framework handles business logic, authentication, data processing, and database access.

However, React also has limitations. Unlike Angular, React is not a complete framework by itself. Additional libraries are normally required for routing, advanced state management, form handling, data fetching, and interface components. This flexibility can become a weakness because poor library selection may lead to inconsistent project structure and higher maintenance difficulty. React's ecosystem also changes frequently, so developers must keep track of current best practices and compatible library versions. For MyHealer, these weaknesses can be controlled by limiting the number of external libraries and using stable, widely adopted tools only when necessary. Overall, React provides the best balance between flexibility, component reuse, ecosystem support, and development feasibility, making it the most suitable front-end web framework for MyHealer.

2.1.5 Back-end Framework

The back-end framework is a critical component of MyHealer because it acts as the central layer that connects the patient mobile application, staff web dashboard, hospital database, and AI-assisted fracture classification service. While the mobile and web front ends provide the interfaces used by patients, doctors, staff, pharmacists, and administrators, the back end is responsible for processing requests, enforcing business rules, managing user access, communicating with the database, and exposing application programming interfaces for the front-end clients. In MyHealer, the back-end framework must therefore support hospital workflows such as appointment management, queue handling, patient registration, medical record access, prescription processing, pharmacy operations, staff administration, hospitalisation management, and integration with the AI prediction service.

A suitable back-end framework for MyHealer must satisfy several requirements. First, it must support structured data management because hospital operations depend on highly related data entities such as patients, doctors, appointments, prescriptions, radiology records, payments, wards, beds, and staff roles. Second, it must support authentication and authorisation because different users must only access functions permitted by their roles. For example, patients should not access doctor diagnosis functions, pharmacists should not manage hospital staff accounts, and administrative users should not directly modify AI model outputs without proper workflow controls. Third, the back end must expose stable REST APIs so that the Flutter mobile application and React web dashboard can communicate with the system consistently. Lastly, because MyHealer includes an AI-assisted fracture classification module developed in Python, a Python-based back-end framework is advantageous because it reduces language fragmentation between the hospital management system and the AI service.

Several back-end framework options were considered for MyHealer, namely Django with Django REST Framework, Laravel, Express.js, and FastAPI. Django is a high-level Python web framework designed for rapid development and clean design (Django Software Foundation, n.d.-a), while Django REST Framework is a toolkit for building Web APIs on top of Django (Encode OSS Ltd., n.d.-a). Laravel is a PHP web application framework that provides built-in support for common web application functions such as authentication and validation (Laravel, n.d.-a; Laravel, n.d.-b). Express.js is a minimal and flexible Node.js web framework for web and mobile applications (OpenJS Foundation, n.d.). FastAPI is a modern

Python framework for building APIs using Python type hints (FastAPI, n.d.). Table 2.5 summarises the comparison of these frameworks.

Table 2.5: A Summarized Comparison of Back-end Frameworks

Aspect	Django + Django REST Framework (Django Software Foundation, n.d.-a; Encode OSS Ltd., n.d.-a)	Laravel (Laravel, n.d.-a)	Express.js (OpenJS Foundation, n.d.)	FastAPI (FastAPI, n.d.)
Main language	Python	PHP	JavaScript	Python
Framework type	Full-stack web framework with REST API toolkit	Full-stack MVC web framework	Minimal web framework	API-focused framework
Data management	Built-in model layer and ORM	Eloquent ORM	Requires additional database libraries	Requires additional database libraries
Authentication and access control	Built-in authentication, groups, permissions, and session support	Built-in authentication support	Usually added through middleware and external packages	Usually added through dependencies and external packages
API development	Strong REST API support through serializers, viewsets, permissions, and authentication	API support available but requires Laravel-specific implementation	Flexible API development through routes and middleware	Strong API development with type hints and automatic documentation

Strengths	Suitable for structured hospital workflows; strong ORM; built-in admin; strong authentication; Python-based; integrates well with AI stack	Productive full-stack development; strong Laravel ecosystem; useful validation and authentication features	Lightweight; flexible; suitable for JavaScript-based applications; large Node.js ecosystem	Fast API development; high performance; Python-based; suitable for AI model serving
Weaknesses	Heavier than microframeworks; may require careful project structure as the system grows	PHP does not align with the Python AI stack; separate language ecosystem from AI module	Requires more manual architecture decisions; many core features depend on external packages	Less complete for full hospital-management workflows because it lacks Django-style built-in admin and full-stack features
Suitability for MyHealer	High	Medium	Medium	Medium for main back end, High for AI prediction service

Django with Django REST Framework was selected as the most suitable back-end framework for MyHealer. Django is appropriate because the system contains many structured and relational workflows, including patient registration, doctor consultation, appointment scheduling, prescription handling, pharmacy inventory, billing, ward admission, and discharge management. Django's model layer allows each data entity to be represented as a Python class that maps to a database table, and it provides an automatically generated database-access API for creating, retrieving, updating, and deleting records (Django Software Foundation, n.d.-b). This is suitable for MyHealer because the system requires consistent

handling of related hospital records rather than loosely structured data.

Another major advantage of Django is its built-in authentication and permission support. Django's authentication system handles user accounts, groups, permissions, and user sessions (Django Software Foundation, n.d.-c). This is important for MyHealer because the system includes multiple user groups, namely patients, doctors, hospital staff, pharmacists, and administrators. Each group requires different access rights. For example, doctors require access to diagnosis and AI fracture classification results, pharmacists require access to prescription processing and medication distribution, and administrators require access to staff and reporting functions. By using Django's authentication and permission mechanisms, role-based access control can be implemented more consistently than if authentication had to be built from scratch.

Django also provides an automatic admin interface, which is one of its most useful features for administrative systems (Django Software Foundation, n.d.-d). Although MyHealer will still require custom dashboards for hospital users, the Django admin interface can support development, testing, and internal data verification during the project. This is useful for a solo developer because records such as users, appointments, prescriptions, queue entries, and ward data can be inspected and managed during implementation without first building every custom administrative screen. In addition, Django provides built-in security features and guidance covering common web security concerns, including input validation and secure handling of user-controlled data (Django Software Foundation, n.d.-e). This is important because hospital systems handle sensitive patient information and must avoid careless treatment of personal and medical data.

Django REST Framework strengthens Django by adding dedicated tools for API development. It provides serializers, authentication support, permissions, and viewsets for building structured Web APIs (Encode OSS Ltd., n.d.-a; Encode OSS Ltd., n.d.-b). This is directly aligned with MyHealer's architecture because the Flutter mobile application and React web dashboard must communicate with the back end through APIs. For example, the patient application must request appointment details, queue positions, medical records, and payment information. The doctor dashboard must request patient histories, upload radiology images, receive AI prediction results, and save diagnosis records. The hospital staff dashboard must manage patient registration, appointment queues, and radiology image records. Django REST Framework provides a suitable API layer for these interactions while

allowing the core business logic and database models to remain within Django.

Laravel is also a strong back-end framework. It provides productive web application development features, including authentication and validation support (Laravel, n.d.-a; Laravel, n.d.-b). However, Laravel is PHP-based, while MyHealer's AI module is developed using Python. This difference would introduce an additional programming language into the system, increasing integration complexity. The AI-assisted fracture classification model will be trained using PyTorch and served through a Python-based API, so Django provides better language alignment with the AI component. Although Laravel could support the hospital management features, it does not provide the same direct development consistency with the Python AI ecosystem.

Express.js provides another possible option. It is lightweight and flexible, and it is suitable for building APIs and web applications in the Node.js ecosystem (OpenJS Foundation, n.d.). However, this flexibility also means that many important functions, such as authentication, role management, database modelling, validation, and administration, usually require additional libraries and architectural decisions. For a solo FYP developer, this can increase development risk because more time must be spent selecting, configuring, and maintaining separate packages. Since MyHealer requires structured hospital workflows and many role-based functions, a more complete back-end framework is more suitable than a minimal framework.

FastAPI is highly relevant to MyHealer because it is Python-based and suitable for building high-performance APIs (FastAPI, n.d.). However, it is more appropriate for the AI prediction service than for the main hospital management back end. The AI service mainly needs to receive an X-ray image, run the trained CNN model, produce a fractured or non-fractured prediction, generate a confidence score, and return the Grad-CAM result. FastAPI is well suited to this narrower prediction API. In contrast, the main hospital management system requires broader full-stack support, including relational data modelling, built-in administration, authentication, permissions, and management of multiple business workflows. For this reason, FastAPI is retained as the AI-serving framework, while Django with Django REST Framework is selected as the main back-end framework.

The main weakness of Django is that it is heavier than a microframework. A small application with only a few API endpoints may be implemented more quickly using

Express.js or FastAPI. Django also requires careful project organisation because models, serializers, views, permissions, and business logic can become difficult to maintain if the system grows without a clear structure. However, this weakness is acceptable for MyHealer because the system is not a simple API-only application. It includes multiple modules, user roles, relational records, administrative functions, and integration with an AI service. Therefore, Django with Django REST Framework provides the best balance of structured development, security support, API capability, Python alignment, and feasibility for the MyHealer project.

2.1.6 Database

The database is a fundamental component of MyHealer because it stores and organises the data required by the hospital management platform and the AI-assisted fracture classification workflow. MyHealer involves several interconnected data entities, including patients, doctors, hospital staff, pharmacists, administrators, appointments, queue records, medical records, X-ray image metadata, AI prediction results, prescriptions, medication inventory, payments, ward admissions, beds, discharges, and billing records. Therefore, the selected database must be able to manage structured relational data reliably while supporting secure and consistent access by the Django back end, Flutter mobile application, React web dashboard, and FastAPI prediction service.

A suitable database for MyHealer must satisfy several requirements. First, it must support relational data modelling because hospital data is highly connected. For example, one patient may have many appointments, one appointment may produce one or more consultation records, a consultation may lead to prescriptions and X-ray analysis results, and a hospitalisation record may be linked to ward, bed, billing, and discharge information. Second, the database must support data integrity because medical and administrative records cannot be treated as loosely connected information. Relationships between records must be protected through primary keys, foreign keys, constraints, and transactions. Third, the database must support scalability and long-term maintainability because the system is expected to handle growing numbers of patients, records, and diagnostic results. Lastly, the database should align well with Django, since Django is selected as the main back-end framework for MyHealer.

Several database options were considered for MyHealer, namely PostgreSQL, MySQL, MongoDB, and SQLite. PostgreSQL is an open-source object-relational database system with a long development history and support for ACID-compliant transactions (PostgreSQL Global Development Group, n.d.-a). MySQL is a widely used relational database management system and is commonly used for web applications (Oracle, n.d.). MongoDB is a document database that stores data in flexible JSON-like documents rather than traditional relational tables (MongoDB, n.d.). SQLite is a self-contained, serverless, transactional SQL database engine suitable for lightweight applications and embedded use (SQLite, n.d.). Table 2.6 summarises the comparison of these database options.

Table 2.6: A Summarized Comparison of Databases

Aspect	PostgreSQL (PostgreSQL Global Development Group, n.d.-a)	MySQL (Oracle, n.d.)	MongoDB (MongoDB, n.d.)	SQLite (SQLite, n.d.)
Database type	Object-relational database	Relational database	NoSQL document database	Serverless relational database
Data model	Tables, relationships, SQL, JSON support	Tables, relationships, SQL	Flexible JSON-like documents	Tables and SQL in a local database file
Transaction support	Strong ACID support	Transaction support through storage engines such as InnoDB	Transaction support available, but document-oriented design remains its main model	Transactional, but more suitable for local or lightweight workloads
Relationship handling	Strong support through primary keys, foreign keys, constraints, and joins	Strong relational support	Relationships are usually handled through embedded documents or	Supports relational tables but is limited for multi-user server workloads

			application logic	
Django compatibility	Officially supported and feature-rich in Django	Officially supported by Django	Requires third-party integration for Django	Officially supported by Django, mainly useful for development or small applications
Strengths	Strong relational modelling; ACID compliance; extensibility; JSON support; good fit for Django	Mature and widely used; suitable for many web applications; strong community	Flexible schema; useful for semi-structured and rapidly changing data	Simple setup; no separate database server required; suitable for prototyping
Weaknesses	Requires database server setup and administration	Less advanced feature set than PostgreSQL for some complex data types and extensions	Less suitable for strongly relational hospital records; weaker fit with Django's default ORM	Not ideal for production systems with many concurrent users
Suitability for MyHealer	High	Medium	Low to Medium	Low for production, Medium for early development

PostgreSQL was selected as the most suitable database for MyHealer. The primary reason is that MyHealer requires strong relational data modelling. Hospital records are not isolated documents; they are connected through clear relationships. A patient's account is connected to appointments, consultations, medical records, prescriptions, payments, AI prediction results, and hospitalisation records. A relational database is therefore more appropriate than a document-oriented database because it can enforce relationships using foreign keys and constraints. PostgreSQL supports this relational model while also providing ACID-compliant transaction behaviour, which is important for preserving consistency when sensitive records are created or updated (PostgreSQL Global Development Group, n.d.-a).

PostgreSQL is also well aligned with Django. Django officially supports several database back ends, including PostgreSQL, MariaDB, MySQL, Oracle, and SQLite (Django Software Foundation, n.d.-f). However, Django also provides PostgreSQL-specific support for data types and features, reflecting PostgreSQL's strong feature set among Django-supported databases (Django Software Foundation, n.d.-g). This makes PostgreSQL a natural fit for MyHealer because the Django back end will manage most of the system's business logic and data access. By using PostgreSQL with Django's object-relational mapping layer, the system's models, such as Patient, Appointment, Prescription, RadiologyRecord, PredictionResult, Ward, Bed, and Payment, can be mapped clearly to database tables.

Another advantage of PostgreSQL is its ability to support both structured relational data and selected semi-structured data. PostgreSQL includes JSON and JSONB data types, with JSONB designed for more efficient processing than plain JSON storage (PostgreSQL Global Development Group, n.d.-c). This is useful for MyHealer because most hospital records should be stored relationally, but some AI-related outputs may contain flexible metadata. For example, the database may store the prediction label, confidence score, Grad-CAM image path, model version, upload timestamp, and processing status as structured fields, while additional AI metadata can be stored in a JSONB field if needed. This allows the system to remain relational while still supporting flexible storage for AI prediction details.

MySQL was considered because it is mature, widely used, and suitable for many web-based systems. It also supports transactions through the InnoDB storage engine, including standard transaction isolation levels (Oracle, n.d.). However, PostgreSQL is more suitable for MyHealer because it provides stronger alignment with Django's advanced database features and offers richer support for extensibility and complex data types. Since MyHealer involves

medical records, diagnostic metadata, role-based workflows, and reporting, PostgreSQL provides a more flexible foundation for long-term development.

MongoDB was also considered because its flexible document model is useful when data structures change frequently. MongoDB stores data as JSON-like documents, which allows developers to evolve data models without the rigid structure of relational tables (MongoDB, n.d.). However, this flexibility is not the main requirement of MyHealer. Hospital data is highly structured and relationship-heavy, so a document database would place more responsibility on the application layer to maintain relationships and consistency. In addition, MongoDB is not one of Django's officially supported database back ends, so it would require third-party integration and may increase development complexity. For these reasons, MongoDB is less suitable as the main database for MyHealer.

SQLite was considered because it is lightweight, serverless, and easy to set up. It is useful during early development because a developer can create and test database models without configuring a separate database server. SQLite is also officially supported by Django (Django Software Foundation, n.d.-f). However, SQLite is not ideal for MyHealer's production-level design because the system involves multiple users, concurrent access, hospital workflows, and potentially growing volumes of medical and diagnostic records. Therefore, SQLite may be useful for small prototypes or local testing, but it is not suitable as the main production database.

The main weakness of PostgreSQL is that it requires more setup and administration than SQLite. A PostgreSQL server must be installed, configured, backed up, and maintained. This may increase the early setup effort for a solo developer. However, this weakness is acceptable because MyHealer is not a simple local application. It is an integrated hospital management platform with AI-assisted diagnostic support, multiple user roles, relational data, medical records, prescriptions, payments, and hospitalisation workflows. PostgreSQL provides the strongest balance of reliability, relational integrity, Django compatibility, extensibility, and future scalability, making it the most suitable database for MyHealer.

2.1.7 Application Programming Interface Architectural Style

An application programming interface, commonly known as an API, is required in MyHealer because the system is developed using separate front-end, back-end, database, and AI components. The patient mobile application is developed using Flutter, while the staff, doctor, pharmacist, and administrator dashboards are developed using React. These front-end clients do not communicate directly with the PostgreSQL database. Instead, they send requests to the Django back end, which applies business rules, validates user permissions, retrieves or updates database records, and returns structured responses. In addition, the AI-assisted fracture classification module is served through a separate prediction service, where an X-ray image is submitted, processed by the trained CNN model, and returned with a prediction result, confidence score, and Grad-CAM output. Therefore, a suitable API architectural style is needed to standardise communication between these components.

The API architectural style selected for MyHealer must satisfy several requirements. First, it must be simple enough to support rapid development by a solo developer. Second, it must support common create, read, update, and delete operations because the system contains many resource-based workflows, including patients, appointments, queue records, prescriptions, payments, radiology records, AI prediction results, wards, beds, and discharge records. Third, it must be compatible with both Flutter and React because both front ends need to exchange data with the back end. Fourth, it must integrate well with Django REST Framework because Django has been selected as the main back-end framework. Lastly, it must be suitable for hospital management functions where clear endpoint design, predictable request handling, and maintainable integration are more important than highly specialised data-query or streaming mechanisms.

In MyHealer, REST APIs are also used to support the real-time queue monitoring feature. Queue records can be represented as resources, where the patient mobile application retrieves the current queue number, estimated position, and appointment status from the back end. Hospital staff can also update queue progress through the web dashboard, allowing patients to receive updated queue information through the mobile application. Since the queue feature is a supporting function rather than the core AI contribution, it does not require a separate specialised API architecture. A REST-based design is sufficient because the queue workflow mainly involves retrieving, updating, and displaying queue-related resources.

Four common API architectural styles were considered for MyHealer: REST, GraphQL, SOAP, and gRPC. REST was introduced by Fielding as an architectural style for distributed hypermedia systems, with a uniform interface as one of its defining constraints (Fielding, 2000). GraphQL is an open-source query language for APIs and server-side runtime that uses a strongly typed schema to define relationships between data (GraphQL Foundation, n.d.). SOAP is a W3C messaging framework for exchanging structured information in decentralised and distributed environments (World Wide Web Consortium, 2007). gRPC is an API framework that commonly uses Protocol Buffers as both its interface definition language and message interchange format (gRPC Authors, n.d.). Table 2.7 summarises the comparison of these API architectural styles.

Table 2.7: A Summarized Comparison of API Architectural Styles

Aspect	REST (Fielding, 2000)	GraphQL (GraphQL Foundation, n.d.)	SOAP (World Wide Web Consortium, 2007)	gRPC (gRPC Authors, n.d.)
Main concept	Resource-based architectural style using a uniform interface	Query language and runtime for flexible API data retrieval	XML-based messaging framework	Remote procedure call framework using Protocol Buffers
Data format	Commonly JSON over HTTP	JSON-like query and response structure	XML	Protocol Buffers
Communication style	Client requests resources through standard HTTP methods	Client specifies the exact data required through a query	Client and server exchange structured XML messages	Client calls defined remote procedures

Strengths	Simple to understand; widely adopted; compatible with web and mobile clients; suitable for CRUD operations; strong fit with Django REST Framework	Flexible data fetching; reduces over-fetching and under-fetching; useful when clients require highly customised responses	Formal structure; suitable for enterprise messaging; supports standards-based message exchange	High performance; strong typing; suitable for internal microservices and real-time communication
Weaknesses	May require multiple endpoints or requests for complex nested data	Higher learning curve; requires schema and resolver design; query complexity must be controlled	Verbose XML messages; heavier and more complex than modern web API styles	Less straightforward for browser-based clients; requires Protocol Buffers and generated client code
Suitability for MyHealer	High	Medium	Low	Medium for internal AI or microservice communication, Low for main client-facing API

REST was selected as the most suitable API architectural style for MyHealer. The main reason is that MyHealer is largely resource-based. Most system functions can be represented clearly as resources, such as patients, doctors, appointments, queue records, medical records, prescriptions, medications, payments, wards, beds, radiology images, and prediction results.

REST fits this structure because resources can be accessed through predictable endpoints and manipulated using standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE. This resource-oriented structure is easier to understand and maintain than a more specialised API style, especially for a solo developer working within the fixed FYP timeline.

REST also aligns strongly with Django REST Framework. Django REST Framework is specifically designed for building Web APIs on top of Django, and it provides features such as serializers, authentication, permissions, and viewsets for REST-oriented API development (Encode OSS Ltd., n.d.). This is important for MyHealer because the back end must expose APIs for both the Flutter mobile application and React web dashboard. For example, the mobile application may request the patient's appointments, queue number, medical record, or payment history, while the web dashboard may request patient information, uploaded X-ray records, AI prediction results, prescriptions, inventory records, or ward availability. By using REST, these functions can be organised into clear and consistent API endpoints.

REST is also suitable because MyHealer does not require the level of query flexibility that GraphQL provides. GraphQL allows clients to request exactly the data they need, which can be useful in systems where different clients require many customised combinations of nested data (GraphQL Foundation, n.d.). However, MyHealer's API requirements are more predictable. Patients, doctors, staff, pharmacists, and administrators use defined workflows rather than freely composing complex queries. Introducing GraphQL would require additional schema design, resolver implementation, query validation, and security control. This would increase development complexity without providing a necessary benefit for the current project scope.

SOAP was considered but is less suitable for MyHealer. SOAP is a formal messaging framework for exchanging structured information using XML (World Wide Web Consortium, 2007). It can be appropriate for legacy enterprise environments or systems requiring strict messaging standards. However, MyHealer is a modern web and mobile application built with Flutter, React, Django REST Framework, PostgreSQL, and FastAPI. SOAP would introduce verbose XML messages and heavier integration requirements. For this project, such complexity is unnecessary because the system mainly requires straightforward communication between front-end clients, the hospital back end, and the AI service.

gRPC was also considered because it provides efficient remote procedure calls and

commonly uses Protocol Buffers for defining services and exchanging messages (gRPC Authors, n.d.). This makes gRPC useful for high-performance internal communication between services, especially in microservice architectures. However, MyHealer's main API is consumed by browser-based and mobile clients. For these clients, REST is more straightforward because JSON over HTTP is easier to integrate, inspect, debug, and test during development. gRPC may be considered in future if MyHealer becomes a larger distributed system with multiple internal services, but it is not the most suitable choice for the main client-facing API at the current prototype stage.

The main weakness of REST is that complex nested data may require multiple requests or carefully designed endpoints. For example, a doctor dashboard may need patient details, appointment information, radiology records, AI prediction results, and prescriptions on the same screen. If the endpoints are designed too narrowly, the front end may need to make several API calls to display one complete view. However, this weakness can be controlled by designing appropriate endpoint structures and using selected combined response endpoints where necessary. Overall, REST provides the best balance of simplicity, compatibility, maintainability, Django REST Framework support, and suitability for MyHealer's resource-based hospital workflows.

2.1.8 Deep Learning for Bone Fracture Classification

The deep learning component is the central contribution of MyHealer. Although the system also includes hospital management functions such as appointment handling, queue monitoring, consultation management, prescription processing, pharmacy management, administration, and hospitalisation, these functions act as the supporting platform for the AI-assisted diagnostic feature. The main technical focus of the project is the classification of X-ray images into two categories, namely fractured and non-fractured. This classification result is intended to support doctors during radiographic review by providing an additional AI-assisted opinion, not to replace the professional judgement of medical practitioners.

In MyHealer, the fracture classification process begins when an X-ray image is uploaded through the doctor module. The image is then processed by a trained convolutional neural network, which produces a prediction indicating whether the image is classified as fractured or non-fractured. The system also displays a confidence score, which represents the model's probability-based certainty for the predicted class. In addition, Gradient-weighted Class Activation Mapping, commonly known as Grad-CAM, is used to generate a heatmap that highlights the image regions most influential to the model's prediction (Selvaraju et al., 2017). This improves the interpretability of the AI output by allowing the doctor to visually inspect whether the model appears to focus on medically relevant areas of the X-ray image.

The selected AI task is binary image classification. This means that the system does not attempt to classify specific fracture types, detect exact fracture boundaries, or generate bounding boxes around fractured areas. This scope is suitable for MyHealer because the selected dataset, Bone Fracture Multi-Region X-ray Data, is organised into fractured and non-fractured image folders across training, validation, and testing splits. Since the dataset provides image-level class labels rather than object-level annotations, a classification-based CNN is more appropriate than an object detection model. The requirement to highlight the suspected fracture area is addressed using Grad-CAM, which provides a visual explanation of the model's attention without requiring bounding box-labelled data.

Deep learning is suitable for this task because bone fracture patterns in X-ray images can be visually subtle, irregular, and affected by anatomical region, image quality, patient positioning, and overlapping bone structures. Traditional image processing techniques depend heavily on manually designed operations such as edge detection, thresholding, and contour

extraction, while conventional machine learning methods usually require hand-crafted features before classification. In contrast, deep learning models can learn hierarchical image features directly from training data, making them particularly effective for image recognition tasks (LeCun et al., 2015). For MyHealer, this allows the model to learn visual patterns related to fractured and non-fractured X-ray images without relying fully on manually designed image rules.

The use of deep learning in medical image analysis is supported by existing research. Aggarwal et al. (2021) reported that deep learning methods have demonstrated clinically acceptable diagnostic accuracy across different medical imaging applications, although they also emphasised the importance of proper validation and transparent reporting. In fracture-related studies, deep learning has been shown to assist clinicians in radiograph interpretation. Lindsey et al. (2018) found that a deep neural network improved clinician performance in wrist fracture detection, while Duron et al. (2021) reported that AI assistance improved the detection of appendicular skeletal fractures by both radiologists and emergency physicians. These studies support the feasibility of using AI as a diagnostic aid, but they also indicate that human oversight remains essential.

Therefore, the deep learning module in MyHealer is positioned as a diagnostic support tool. It provides a predicted class, confidence score, and Grad-CAM heatmap to assist doctors during X-ray review, while the final decision remains under the doctor's responsibility. This positioning is important because the model is developed as an academic prototype using a public dataset, and it has not undergone external clinical validation, regulatory assessment, or real-hospital deployment testing. As a result, the AI module should be evaluated as a proof-of-concept system that demonstrates how convolutional neural networks can be integrated into a hospital management platform to support bone fracture classification.

This section reviews the deep learning concepts, model architecture choices, existing fracture detection studies, dataset selection, interpretability method, AI training framework, and model serving framework used in MyHealer. The review provides the theoretical and technical foundation for the development of the CNN-based fracture classification module, which is the core technology of the project.

2.1.8.1 Convolutional Neural Network

The selection of a suitable image analysis approach is important because MyHealer relies on X-ray image interpretation as its core AI function. Bone fracture classification is not a simple visual task because fractures may appear as subtle lines, discontinuities, or abnormal bone patterns, and these visual features may vary depending on the anatomical region, image quality, patient positioning, and overlap of bone structures. Therefore, the system requires an approach that can learn meaningful image features from X-ray data rather than depending only on fixed manual rules.

Three image analysis approaches were considered for MyHealer: traditional image processing, conventional machine learning, and deep learning using convolutional neural networks. Traditional image processing uses manually designed operations such as edge detection, thresholding, contour detection, and texture extraction. Conventional machine learning uses algorithms such as support vector machines, decision trees, random forests, or artificial neural networks after features have been manually extracted from the image. Deep learning using CNNs learns visual features automatically from training images, reducing the need for manual feature engineering (LeCun et al., 2015). Table 2.8 summarises the comparison of these approaches.

Table 2.8: A Summarized Comparison of Image Analysis Approaches for Bone Fracture Classification

Aspect	Traditional Image Processing	Conventional Machine Learning	Deep Learning using CNN (LeCun et al., 2015)
Main concept	Uses manually designed image operations such as thresholding, edge detection, contour extraction, and texture analysis	Uses classification algorithms with manually extracted image features	Learns visual features automatically from image data using convolutional layers

Feature extraction	Fully hand-crafted	Mostly hand-crafted before training	Automatically learned during training
Strengths	Easier to understand; lower computational requirement; suitable for simple and clear image patterns	More adaptable than fixed image processing rules; can learn decision patterns from extracted features	Strong ability to learn hierarchical visual patterns; suitable for complex image recognition tasks; less dependent on manual feature design
Weaknesses	Sensitive to noise, image quality, and anatomical variation; difficult to generalise across different body regions	Performance depends heavily on the quality of manually selected features; feature engineering can be time-consuming	Requires labelled training data, model tuning, and sufficient computing resources; may be difficult to interpret without explanation methods
Suitability for MyHealer	Low	Medium	High

Based on the comparison in Table 2.8, deep learning using CNN is the most suitable approach for MyHealer. Traditional image processing is not selected because fracture appearances are inconsistent across X-ray images. A fixed edge or threshold rule may work for a small number of clear images, but it is unlikely to generalise well across multiple body regions and different imaging conditions. Conventional machine learning is more flexible than rule-based image processing, but it still depends heavily on manually designed features. If important fracture-related features are not extracted properly, the classifier may not

perform well.

A CNN is more suitable because it can learn image features directly from the X-ray data. In a CNN, convolutional layers apply filters to the input image to detect visual patterns. Earlier layers commonly learn simple visual features such as edges, lines, and contrast changes, while deeper layers can learn more complex patterns related to object structure and class differences (LeCun et al., 2015). This layered feature-learning process is suitable for X-ray classification because fracture-related patterns may be subtle and distributed across different parts of the image.

For MyHealer, the CNN receives an X-ray image as input and produces a binary classification output: fractured or non-fractured. The model is trained using labelled images from the selected Bone Fracture Multi-Region X-ray Data dataset. During training, the CNN adjusts its internal weights by comparing its prediction with the correct class label. Over many training iterations, the model learns visual patterns that help distinguish fractured images from non-fractured images. Once trained, the CNN can be used in the doctor module to provide an AI-assisted prediction for newly uploaded X-ray images.

The suitability of CNNs for medical image analysis is supported by existing literature. Deep learning has been widely applied in medical imaging because it can learn complex visual representations from image data, and a systematic review by Aggarwal et al. (2021) found that deep learning systems have demonstrated clinically acceptable diagnostic accuracy across several medical imaging tasks. In fracture-related research, deep neural networks have also been shown to support radiographic interpretation. For example, Lindsey et al. (2018) found that deep learning assistance improved clinician performance in wrist fracture detection, while Duron et al. (2021) reported that AI assistance improved fracture detection performance for both radiologists and emergency physicians.

However, CNNs also have limitations. First, they require labelled training data, and model performance depends on the quality and representativeness of the dataset. If the dataset does not represent real hospital X-ray conditions well, the trained model may perform poorly on new images. Second, CNNs can overfit if the model learns patterns that are specific to the training data but not generalisable to unseen images. Third, CNN predictions can be difficult to interpret because the model does not naturally explain why a particular class was selected. For this reason, MyHealer includes Grad-CAM as an interpretability technique, allowing

doctors to view a heatmap of the regions that influenced the model's prediction.

In conclusion, a CNN is selected as the core deep learning approach for MyHealer because it is more suitable than traditional image processing and conventional machine learning for multi-region X-ray fracture classification. Its ability to learn hierarchical image features directly from labelled X-ray data supports the binary classification goal of the project. Although CNNs have limitations related to data dependency, overfitting, and interpretability, these issues can be reduced through appropriate preprocessing, data augmentation, validation monitoring, and Grad-CAM visualisation. Therefore, CNN is considered the most appropriate foundation for the AI-assisted bone fracture classification module in MyHealer.

2.1.8.2 CNN Architectures for Image Classification

After selecting convolutional neural networks as the most suitable deep learning approach, the next decision concerns the type of CNN architecture to be used for MyHealer. A CNN architecture defines how the convolutional layers, activation functions, pooling layers, regularisation layers, and classification layers are arranged. This decision is important because the architecture affects the model's learning ability, training complexity, computational cost, interpretability, and suitability for the selected dataset.

Several CNN architecture options were considered for MyHealer, including a custom CNN trained from scratch, VGG, ResNet, DenseNet, and EfficientNet. These architectures represent different levels of complexity. VGG introduced a deep but simple architecture based on repeated small convolution filters (Simonyan & Zisserman, 2015). ResNet introduced residual learning to make very deep networks easier to train (He et al., 2016). DenseNet improved feature reuse by connecting each layer to later layers (Huang et al., 2017). EfficientNet introduced compound scaling, where model depth, width, and input resolution are scaled together in a balanced way (Tan & Le, 2019). A custom CNN, in contrast, allows the architecture to be designed according to the project scope, dataset size, available computing resources, and explanation requirements.

Table 2.9 summarises the comparison of the CNN architecture options considered for MyHealer.

Table 2.9: A Summarized Comparison of CNN Architecture Options

Aspect	Custom CNN (LeCun et al., 2015)	VGG (Simonyan & Zisserman, 2015)	ResNet (He et al., 2016)	DenseNet (Huang et al., 2017)	EfficientNet (Tan & Le, 2019)

Main concept	A CNN architecture designed and trained from scratch for the selected dataset	A deep CNN using repeated small convolution filters	A deep CNN that uses residual connections to improve training	A CNN that connects each layer to later layers for feature reuse	A CNN family that scales depth, width, and image resolution together
Development complexity	Low to Medium	Medium	Medium to High	High	High
Training requirement	Can be adjusted based on available dataset and computing resources	Requires more computation because of a large number of parameters	Requires more training resources for deeper variants	Can be computationally demanding due to dense connections	Requires careful scaling and configuration
Strengths	Full control over architecture; easier to explain; suitable for a from-scratch FYP implementation; can be kept	Simple and well-known structure; useful baseline for image classification	Strong performance in image recognition; residual learning supports deeper models	Strong feature reuse; improved gradient flow; efficient learning of feature representations	Strong balance between accuracy and efficiency in modern image classification

	lightweight				
Weaknesses	May perform below large pre-trained models; requires tuning; risk of overfitting	Large parameter size; higher memory and computation cost	More complex than a simple CNN; may be excessive for a prototype	More difficult to implement and explain clearly; higher architectural complexity	More difficult to explain and tune; may reduce transparency for an academic prototype
Suitability for MyHealer	High	Medium	Medium	Medium	Medium

Based on Table 2.9, a custom CNN trained from scratch is selected as the most suitable architecture for MyHealer. This choice is aligned with the project scope because MyHealer performs binary classification, where each X-ray image is classified as either fractured or non-fractured. The selected dataset already provides labelled image folders for training, validation, and testing, so a custom CNN can be designed to learn directly from these image labels without requiring pre-trained weights or object detection annotations.

A custom CNN is also suitable because it supports academic transparency. Since MyHealer is an FYP project, the model should not only produce a result, but also be explainable in the report. A custom CNN allows the architecture to be described clearly, including the purpose of convolutional layers, pooling layers, dropout layers, activation functions, and the final classification layer. This makes it easier to justify how the model learns from X-ray images and how the classification output is produced. In contrast, larger architectures such as ResNet, DenseNet, and EfficientNet may achieve stronger results in advanced research settings, but their internal structures are more complex and may be more difficult to explain within the scope of a solo undergraduate project.

VGG was considered because its architecture is relatively simple compared with later models and has been widely used as a reference architecture in image classification. However, VGG models contain many parameters, which can increase memory usage and training time (Simonyan & Zisserman, 2015). This makes VGG less suitable for MyHealer's current prototype, especially if development is carried out using limited computing resources.

ResNet was also considered because residual connections allow deeper networks to be trained more effectively (He et al., 2016). This is useful for complex image recognition tasks where very deep architectures are needed. However, MyHealer's current scope is binary classification, not detailed fracture-type classification or object detection. Using ResNet may increase implementation complexity and computational cost beyond what is necessary for the prototype. It may be considered in future work if transfer learning or performance benchmarking is expanded.

DenseNet provides another strong architecture because it encourages feature reuse by connecting each layer to later layers (Huang et al., 2017). This can improve gradient flow and reduce redundant feature learning. However, DenseNet is more architecturally complex than a basic CNN and may be harder to implement, train, and explain clearly. Therefore, it is not selected as the primary architecture for the current MyHealer prototype.

EfficientNet was considered because it provides a strong balance between model accuracy and computational efficiency through compound scaling (Tan & Le, 2019). This makes it a strong option for modern image classification tasks. However, EfficientNet is more suitable when a project aims to compare advanced pre-trained architectures or optimise high-performance classification. Since MyHealer is focused on demonstrating a from-scratch CNN for AI-assisted fracture classification, EfficientNet is more appropriate as a future enhancement than as the primary architecture.

The main weakness of using a custom CNN is that it may not perform as well as a well-trained pre-trained architecture. Models such as ResNet, DenseNet, and EfficientNet have benefited from extensive architecture research and are commonly used in transfer learning. A custom CNN must learn all features from the selected dataset alone, so it may require careful tuning to achieve acceptable performance. Overfitting is also a possible risk, especially if the model becomes too complex for the available training data. This limitation can be reduced through image preprocessing, data augmentation, dropout, early stopping,

validation monitoring, and evaluation using multiple metrics.

In conclusion, a custom CNN is selected for MyHealer because it provides the best balance between suitability, feasibility, explainability, and alignment with the binary classification scope. Although advanced CNN architectures may offer stronger performance in some settings, they introduce additional complexity that is not necessary for the current prototype. A custom CNN allows the AI model to be trained from scratch, explained clearly, integrated with Grad-CAM, and deployed through the MyHealer doctor module as the core AI-assisted fracture classification feature.

2.1.8.3 Existing Studies on Automated Bone Fracture Detection and Classification

Existing studies show that deep learning can support the interpretation of bone X-ray images, especially in fracture detection and classification tasks. These studies are important to MyHealer because they provide evidence that AI-assisted radiograph analysis is technically feasible and clinically useful when used as a support tool. However, they also show that AI models differ significantly in their scope, dataset, anatomical region, model type, and level of clinical validation. Therefore, the review of existing studies is necessary to justify the direction of MyHealer while clearly identifying its limitations as an academic prototype.

Several existing studies were reviewed, including studies on orthopaedic trauma radiographs, wrist fracture detection, appendicular skeletal fracture detection, and broader deep learning applications in medical imaging. Table 2.10 summarises the studies that are most relevant to MyHealer.

Table 2.10: Summary of Existing Studies on AI-assisted Fracture Detection and Classification

Study	Scope	Main contribution	Relevance to MyHealer	Limitation for MyHealer
Olczak et al. (2017)	Orthopaedic trauma radiographs, including wrist, hand, and ankle radiographs	Demonstrated that artificial intelligence could analyse orthopaedic trauma radiographs and perform fracture-related image analysis at a level comparable with humans under the study setting	Supports the feasibility of using AI for fracture-related radiograph analysis	The study used hospital radiographs and selected deep learning networks, while MyHealer uses a public multi-region Kaggle dataset and a from-scratch CNN

Lindsey et al. (2018)	Wrist fracture radiographs	Developed a deep neural network for fracture detection and localisation, and showed that clinician performance improved when AI assistance was provided	Supports the role of AI as a diagnostic assistant to clinicians	The study focused on wrist fractures and used expert-annotated radiographs, while MyHealer performs image-level binary classification across multiple regions
Duron et al. (2021)	Adult appendicular skeletal fractures	Evaluated an AI aid for fracture detection and localisation, and found improved performance for radiologists and emergency physicians	Supports the clinical value of AI-assisted fracture detection in radiology and emergency workflows	The study focused on a clinically evaluated AI aid, while MyHealer is an academic prototype using classification and Grad-CAM visualisation
Aggarwal et al. (2021)	Deep learning in medical imaging across multiple diagnostic domains	Reported that deep learning systems showed clinically acceptable diagnostic accuracy across several medical imaging tasks	Supports the broader use of deep learning in medical image diagnosis	The review is not specific to MyHealer's selected dataset, architecture, or fracture classification scope

Alam et al. (2025)	Multi-region bone fracture X-ray images	Applied deep learning and transfer learning methods to multi-region bone fracture radiographs	Supports the relevance of multi-region fracture X-ray datasets	The modelling approach differs from MyHealer because MyHealer focuses on a custom CNN trained from scratch
---------------------------	---	---	--	--

Olczak et al. (2017) provided early evidence that artificial intelligence could be applied to orthopaedic trauma radiographs. Their study used a large set of wrist, hand, and ankle radiographs and evaluated deep learning algorithms for tasks such as fracture identification, body part classification, laterality recognition, and exam view classification. The study is relevant to MyHealer because it shows that AI can learn useful patterns from orthopaedic X-ray images. However, its scope is not identical to MyHealer. The study used hospital-derived radiographs and evaluated selected deep learning networks, while MyHealer uses a public multi-region dataset and focuses on binary fractured or non-fractured classification using a from-scratch CNN.

Lindsey et al. (2018) developed a deep neural network to detect and localise fractures in wrist radiographs. Their model was trained using radiographs annotated by senior subspecialised orthopaedic surgeons, and the study examined whether AI assistance could improve clinician fracture detection performance. The study found that clinician performance improved when the deep learning model was used as assistance. This is highly relevant to MyHealer because MyHealer is also designed as a doctor-support system rather than a replacement for medical professionals. However, the study focused specifically on wrist fractures and used expert localisation annotations, whereas MyHealer uses image-level labels from a multi-region dataset and applies Grad-CAM for interpretability instead of training an object detection or localisation model.

Duron et al. (2021) assessed an AI aid for detecting adult appendicular skeletal fractures by radiologists and emergency physicians. The study is important because it examined AI assistance in a clinical diagnostic workflow rather than only reporting model performance in isolation. The findings showed that AI assistance improved the fracture detection

performance of both radiologists and emergency physicians. This supports the concept behind MyHealer because the system presents AI predictions to doctors as support during X-ray review. Nevertheless, the AI tool in Duron et al. (2021) was designed for detection and localisation, while MyHealer is limited to binary classification with Grad-CAM visual explanation. Therefore, MyHealer should not claim to provide exact fracture localisation.

Aggarwal et al. (2021) reviewed the diagnostic accuracy of deep learning in medical imaging more broadly. Their review is relevant because it supports the general feasibility of deep learning for diagnostic image analysis. However, the study also highlighted the need for careful validation, transparent reporting, and high-quality evaluation. This limitation is important for MyHealer because a model trained on a public dataset cannot automatically be considered clinically ready. The model must be treated as a prototype and decision-support tool unless it is externally validated using real clinical data and assessed under appropriate medical and regulatory standards.

Alam et al. (2025) is relevant because it focuses on multi-region bone fracture radiographs, which is closer to the selected dataset direction of MyHealer than studies limited to a single body part. This supports the decision to use a dataset that includes multiple anatomical regions rather than only wrist or hip images. However, MyHealer does not adopt a transfer learning approach as the primary implementation. Instead, a custom CNN is trained from scratch so that the core model development process can be explained clearly and aligned with the project requirement. Therefore, Alam et al. (2025) is useful as supporting literature for the dataset and problem domain, but it is not followed directly as the implementation method.

From the reviewed studies, three important observations can be made. First, AI-assisted fracture analysis has been shown to be feasible across different orthopaedic radiograph settings. Second, the strongest clinical value appears when AI is used to assist clinicians rather than replace them. Third, model scope must be clearly defined because fracture classification, fracture detection, fracture localisation, and fracture-type classification are different tasks. This distinction is important for MyHealer. The system performs binary classification only, meaning that it predicts whether an X-ray image is fractured or non-fractured. It does not detect exact fracture boundaries or classify the type of fracture.

The reviewed studies also reveal the gap addressed by MyHealer. Many existing AI fracture studies focus mainly on the AI model itself, while many hospital management systems focus

on administrative workflows without AI-assisted diagnosis. MyHealer combines these two directions by embedding an AI-assisted fracture classification module into a broader hospital management platform. The doctor can upload an X-ray image, receive a fractured or non-fractured prediction, review the confidence score, inspect the Grad-CAM heatmap, and store the result as part of the patient's consultation record. This integration makes the AI output more directly connected to the clinical workflow of the system.

However, the limitations of existing studies also apply to MyHealer. AI models may not generalise well when used with images from different hospitals, different X-ray machines, different patient populations, or different imaging protocols. A model may also produce false positives or false negatives, especially when fracture patterns are subtle or image quality is poor. Therefore, the MyHealer AI module should be described as an academic proof-of-concept and diagnostic support tool. The final diagnosis must remain under the responsibility of the doctor, and future work should include external validation, larger real-world datasets, and clinical evaluation before any actual deployment.

In conclusion, existing studies support the feasibility of AI-assisted bone fracture analysis and justify the use of deep learning in MyHealer. The literature shows that deep learning models can assist fracture detection and improve clinician performance in selected settings. At the same time, the literature also confirms the need for clear task scope, human oversight, dataset quality, and validation. For this reason, MyHealer adopts a controlled binary classification scope using a from-scratch CNN, supported by a confidence score and Grad-CAM heatmap, and positions the AI result as decision support within the doctor module.

2.1.8.4 Dataset for Bone Fracture Classification

The selection of a suitable dataset is a critical decision in the development of MyHealer's AI-assisted bone fracture classification module. Since the model is trained using supervised learning, the dataset must contain labelled X-ray images that clearly indicate whether each image belongs to the fractured or non-fractured class. The dataset must also match the selected project scope. MyHealer performs binary image classification, not fracture-type classification, object detection, or segmentation. Therefore, a dataset with image-level fractured and non-fractured labels is more suitable than a dataset that mainly focuses on bounding boxes or segmentation masks.

Several public musculoskeletal X-ray datasets were considered for MyHealer, including MURA, GRAZPEDWRI-DX, FracAtlas, and Bone Fracture Multi-Region X-ray Data. MURA is a large musculoskeletal radiograph dataset where studies are labelled as normal or abnormal (Rajpurkar et al., 2017). GRAZPEDWRI-DX is a paediatric wrist trauma X-ray dataset that contains wrist trauma images with annotations created by paediatric radiologists (Nagy et al., 2022). FracAtlas is a fracture dataset that includes annotations for classification, localisation, and segmentation tasks (Abdeen et al., 2023). Bone Fracture Multi-Region X-ray Data is a Kaggle dataset containing fractured and non-fractured X-ray images across multiple anatomical regions, organised into train, validation, and test folders (bmadushanirodrigo, n.d.). Table 2.11 summarises the comparison of these datasets.

Table 2.11: A Summarized Comparison of Public X-ray Datasets for Fracture Classification

Aspect	MURA (Rajpurkar et al., 2017)	GRAZPEDWRI-DX (Nagy et al., 2022)	FracAtlas (Abdeen et al., 2023)	Bone Fracture Multi-Region X-ray Data (bmadushanirodrigo, n.d.)
Main focus	Musculoskeletal abnormality	Paediatric wrist trauma and	Fracture classification,	Multi-region binary bone fracture

	detection	fracture detection	localisation, and segmentation	classification
Anatomical coverage	Upper extremity studies	Wrist region	Multiple musculoskeletal regions	Multiple anatomical regions, including lower limb, upper limb, lumbar, hips, and knees
Label type	Normal or abnormal study labels	Image labels and object annotations	Image labels, bounding boxes, and segmentation masks	Fractured and non-fractured image folders
Suitability for binary fractured or non-fractured classification	Medium	Medium	Medium to High	High
Suitability for MyHealer's selected scope	Medium	Low to Medium	Medium	High

Main limitation for MyHealer	Abnormality does not always mean fracture	Limited to paediatric wrist images	More suitable for detection and segmentation experiments	Public dataset quality and generalisation must be evaluated carefully
-------------------------------------	---	------------------------------------	--	---

Based on Table 2.11, Bone Fracture Multi-Region X-ray Data is selected as the most suitable dataset for MyHealer. The main reason is that its structure directly matches the selected AI task. The dataset contains fractured and non-fractured X-ray images and is already organised into training, validation, and testing folders (bmadushaniredrigo, n.d.). This structure supports a clear supervised learning workflow. The training set is used to train the CNN model, the validation set is used to monitor performance during training and support tuning decisions, and the test set is used to evaluate final model performance after training is completed.

The dataset is also suitable because it covers multiple anatomical regions. MyHealer is designed as a hospital management system with AI-assisted fracture classification, so the system should not be limited to only one body part such as the wrist. A multi-region dataset is more aligned with the hospital setting because patients may present with fractures in different parts of the body. By using a dataset that includes different anatomical regions, the prototype can demonstrate a broader fracture classification concept than a single-region model.

MURA was considered because it is a large musculoskeletal radiograph dataset and has been widely referenced in medical imaging research. However, its task is abnormality detection rather than direct fractured or non-fractured classification (Rajpurkar et al., 2017). This distinction is important because an abnormal musculoskeletal X-ray may include conditions other than fracture. Therefore, although MURA is valuable for general musculoskeletal abnormality research, it is less directly aligned with MyHealer's binary fracture classification scope.

GRAZPEDWRI-DX was also considered because it is a public dataset specifically related to fracture detection in paediatric wrist trauma radiographs (Nagy et al., 2022). However, it focuses on paediatric wrist images, while MyHealer is intended to support fracture

classification across multiple anatomical regions. Using a wrist-only dataset would narrow the system scope and conflict with the selected multi-region direction. Therefore, GRAZPEDWRI-DX is not selected as the primary dataset, although it remains useful as a reference for fracture detection research.

FracAtlas was considered because it provides fracture-related annotations for classification, localisation, and segmentation tasks (Abedeen et al., 2023). This makes it useful for projects involving object detection or segmentation. However, MyHealer does not require bounding box or segmentation annotations because the selected task is binary image classification. Using a dataset mainly valued for localisation and segmentation may encourage an unnecessary shift toward object detection, which is outside the locked scope of the project. Therefore, FracAtlas is not selected as the main dataset, although it may be useful for future work if MyHealer is expanded to exact fracture localisation.

The selected Kaggle dataset also supports the decision not to use object detection. Since the dataset provides image-level class labels through fractured and non-fractured folders, the CNN can learn whether an image belongs to the fractured or non-fractured class. However, it cannot directly learn exact fracture coordinates because bounding box annotations are not provided. Therefore, MyHealer should not claim to detect the exact fracture location. Instead, the visual explanation requirement is handled using Grad-CAM, which highlights image regions that influenced the model's classification result.

Before training, the dataset must be preprocessed to ensure consistency. X-ray images may differ in resolution, brightness, contrast, orientation, and anatomical region. Therefore, images should be resized to a fixed input dimension suitable for the CNN, converted into a consistent colour channel format if required, and normalised so that pixel values are represented consistently during training. Data augmentation may also be used to improve model robustness. For example, small rotations, horizontal flipping where medically appropriate, zooming, and brightness adjustment can increase variation in the training data. However, augmentation must be applied carefully because excessive transformation may create unrealistic X-ray appearances and harm model learning.

The dataset must also be evaluated for class balance and performance reliability. If one class contains significantly more images than the other, accuracy alone may become misleading because a model may appear accurate while performing poorly on the minority class. For this

reason, MyHealer should evaluate the trained model using multiple metrics, including accuracy, precision, recall, F1-score, and confusion matrix analysis. Recall is especially important in fracture classification because a false negative result, where a fractured X-ray is classified as non-fractured, may be more serious than a false positive in a diagnostic support context.

Despite its suitability, the selected dataset has limitations. As a public dataset, it may not fully represent local hospital X-ray machines, imaging protocols, patient demographics, or real clinical workflow conditions. The labels are also image-level labels, so the dataset does not provide expert-marked fracture boundaries for localisation. These limitations mean that a model trained on this dataset should not be treated as clinically validated. The dataset is suitable for developing and evaluating an academic prototype, but future deployment would require testing on real hospital data, expert review by medical professionals, ethical approval, and regulatory consideration.

In conclusion, Bone Fracture Multi-Region X-ray Data is selected because it directly supports MyHealer's binary fractured or non-fractured classification task, provides a ready train, validation, and test structure, and covers multiple anatomical regions. Compared with MURA, GRAZPEDWRI-DX, and FracAtlas, it best matches the locked project scope and avoids unnecessary movement into abnormality detection, wrist-only detection, or object localisation. Its limitations are acknowledged, especially the lack of bounding box labels and the need for external validation, but it remains the most appropriate dataset for developing the CNN-based fracture classification prototype in MyHealer.

2.1.8.5 Grad-CAM for Model Interpretability

Although convolutional neural networks are suitable for image classification, one major limitation is that their prediction process can be difficult to interpret. A CNN may classify an X-ray image as fractured or non-fractured, but the output label alone does not explain which part of the image influenced the decision. This is a significant issue in a medical decision-support system because doctors should not be expected to rely only on a class label and confidence score. Instead, the system should provide a visual explanation that allows the doctor to inspect whether the model appears to focus on medically meaningful regions of the X-ray image.

Gradient-weighted Class Activation Mapping, commonly known as Grad-CAM, is selected as the interpretability technique for MyHealer. Grad-CAM generates a visual explanation for a CNN prediction by using the gradient information flowing into the final convolutional layer of the network. This allows the method to identify image regions that are important for a selected target class, producing a heatmap that can be overlaid on the original image (Selvaraju et al., 2017). In MyHealer, this heatmap is used to show the regions that contributed most strongly to the model's fractured or non-fractured prediction.

Grad-CAM is especially suitable for MyHealer because the selected project scope is binary classification, not object detection. The dataset used in MyHealer provides image-level labels through fractured and non-fractured folders, but it does not provide bounding box annotations or segmentation masks for exact fracture locations. Therefore, the CNN cannot be trained to draw an exact box around the fracture area. Grad-CAM solves this limitation at the interpretability level by producing a heatmap based on the trained classification model. This means that the system can still provide a visual indication of the model's focus area without changing the project into an object detection or segmentation task.

In the MyHealer workflow, Grad-CAM is applied after the CNN model produces its prediction. When a doctor uploads an X-ray image, the trained CNN classifies it as fractured or non-fractured and generates a confidence score. Grad-CAM is then used to produce a heatmap for the predicted class. The doctor dashboard can display the original X-ray image, the predicted class, the confidence score, and the Grad-CAM heatmap together. This provides a more meaningful AI-assisted result because the doctor can compare the highlighted area with the original X-ray before making a final diagnosis.

The use of Grad-CAM also supports the objective of making the AI module less like a black box. Without interpretability, the system would only return a prediction, which may be difficult for doctors to trust or evaluate. With Grad-CAM, the doctor can observe whether the model's attention is located near the suspected bone region or whether it appears to focus on irrelevant areas. If the heatmap focuses on an unrelated region, the doctor may treat the AI output with greater caution. This is important because MyHealer is intended to assist professional judgement, not replace it.

However, Grad-CAM also has limitations. The heatmap produced by Grad-CAM is not the same as an exact fracture localisation result. It is usually coarse and only indicates the areas that influenced the model's decision. A highlighted region does not prove that a fracture exists in that exact location, and a region without strong highlighting does not prove that no fracture exists. Therefore, the Grad-CAM output must be interpreted as an explanation of model attention rather than as a medically confirmed fracture boundary.

Another limitation is that Grad-CAM depends on the quality of the trained CNN model. If the model has learned incorrect patterns from the training data, the heatmap may also highlight misleading regions. For example, the model may focus on image artefacts, background patterns, labels, or positioning features instead of actual bone structures. This means that Grad-CAM does not automatically guarantee correct reasoning. It only helps users inspect where the model appears to focus. Therefore, model evaluation, dataset quality checking, and doctor review remain necessary.

In conclusion, Grad-CAM is selected for MyHealer because it provides a practical interpretability method for a CNN-based binary classification system. It allows the system to display a visual heatmap without requiring bounding box-labelled data, which fits the selected dataset and project scope. Although Grad-CAM cannot provide exact fracture localisation, it supports the doctor by showing the image regions that influenced the model's prediction. For this reason, Grad-CAM is suitable as the visual explanation method for MyHealer's AI-assisted bone fracture classification module.

2.1.8.6 AI Framework for Model Training

The AI training framework is an important technology decision for MyHealer because it determines how the convolutional neural network is built, trained, evaluated, saved, and later integrated into the prediction service. Since the core contribution of MyHealer is AI-assisted bone fracture classification, the selected framework must support image preprocessing, tensor computation, neural network construction, loss calculation, optimisation, automatic differentiation, GPU acceleration, model evaluation, and model saving. The framework should also be suitable for implementing a custom CNN trained from scratch because MyHealer does not depend on a pre-trained classification model as its primary implementation.

The training framework must also align with the overall development environment of MyHealer. The main back end is developed using Django and Django REST Framework, while the AI prediction service is served using FastAPI. Since these technologies are Python-based, a Python-compatible deep learning framework is preferred. This reduces language fragmentation and makes it easier to connect the trained model with the AI prediction API. It also allows image preprocessing, model inference, Grad-CAM generation, and API serving to be implemented within the same language ecosystem.

Three AI training framework options were considered for MyHealer: PyTorch, TensorFlow, and Keras. PyTorch is an optimised tensor library for deep learning using CPUs and GPUs (PyTorch, n.d.-a). TensorFlow is an open-source machine learning platform that supports the creation of machine learning models for different environments (TensorFlow, n.d.). Keras is a high-level deep learning API designed to support readable, concise, and maintainable model development (Keras, n.d.). Table 2.12 summarises the comparison of these frameworks.

Table 2.12: A Summarized Comparison of AI Training Frameworks

Aspect	PyTorch (PyTorch, n.d.-a)	TensorFlow (TensorFlow, n.d.)	Keras (Keras, n.d.)
--------	---------------------------	-------------------------------	---------------------

Main concept	Deep learning framework with tensor computation, GPU support, automatic differentiation, and neural network modules	End-to-end machine learning platform for developing and deploying machine learning models	High-level deep learning API focused on readability, simplicity, and fast experimentation
Programming language alignment	Python-based and suitable for integration with FastAPI and other Python services	Python-based with broad machine learning ecosystem support	Python-based and commonly used for rapid model development
Training control	Provides explicit control over training loops, model architecture, loss calculation, and optimisation	Supports both high-level and lower-level training workflows	Provides simplified model building and training APIs
Strengths	Flexible model development; strong Python integration; suitable for custom CNN implementation; clear control over training and evaluation	Comprehensive machine learning ecosystem; strong deployment support; suitable for large-scale machine learning workflows	Simple syntax; beginner-friendly; fast model prototyping; readable model definition
Weaknesses	Requires more manual coding than high-level APIs; training loops and	Can be heavier and more complex for a small prototype; may	Less direct control than lower-level framework usage; may be less

	evaluation steps must be carefully implemented	introduce more tooling than required	suitable when the training process needs to be explained in detail
Suitability for MyHealer	High	Medium	Medium

Based on Table 2.12, PyTorch is selected as the most suitable AI training framework for MyHealer. The main reason is that PyTorch provides a flexible environment for implementing a custom CNN from scratch. In MyHealer, the model architecture must be designed, trained, evaluated, and explained clearly. PyTorch supports this requirement because it allows the developer to define the CNN layers, write the training loop, calculate loss, perform backpropagation, update model weights, and evaluate validation performance in a transparent manner. This makes it suitable for an FYP project where the development of the core AI technology must be understood and justified.

PyTorch is also suitable because it supports automatic differentiation through its autograd system, which is important for neural network training (PyTorch, n.d.-b). During training, the model makes predictions from X-ray images, calculates the difference between the predicted output and the correct label, and updates its internal weights to reduce the error. Automatic differentiation allows gradients to be computed efficiently, which enables the CNN to learn from the dataset. This is essential for MyHealer because the model must learn fracture-related image patterns from labelled X-ray images.

Another advantage of PyTorch is its strong alignment with the Python ecosystem. Since MyHealer uses FastAPI to serve the trained AI model, the PyTorch model can be loaded directly in the FastAPI prediction service. This allows the prediction API to receive an uploaded X-ray image, apply the same preprocessing steps used during training, run model inference, calculate the confidence score, and generate the Grad-CAM heatmap. This direct integration reduces the complexity that may occur if the training framework and serving environment are poorly aligned.

TensorFlow was also considered because it is a mature and widely used machine learning

platform. It supports machine learning model development across different environments and has a broad ecosystem for training and deployment (TensorFlow, n.d.). However, TensorFlow may introduce more tooling and complexity than necessary for the current MyHealer prototype. Since the project focuses on a custom CNN trained from scratch and integrated into a Python-based prediction service, PyTorch provides a more straightforward and flexible option for implementation and explanation.

Keras was considered because it provides a high-level deep learning API with readable and concise model development features (Keras, n.d.). This makes it attractive for rapid experimentation and beginner-friendly model building. However, MyHealer requires enough control over the training process to explain the core AI workflow clearly. A very high-level API may simplify development, but it can hide important training details that should be discussed in an academic report, such as data loading, forward propagation, loss calculation, backpropagation, validation monitoring, and metric evaluation. Therefore, Keras is useful as a reference for rapid prototyping, but PyTorch is more suitable for the main implementation.

The main weakness of PyTorch is that it requires more manual coding than high-level APIs. The developer must carefully implement data loading, preprocessing, model architecture, training loops, validation loops, metric calculation, model saving, and error handling. If these steps are not structured properly, implementation mistakes may affect the reliability of the model. However, this weakness is acceptable for MyHealer because the project aims to demonstrate the AI development process clearly. With proper coding structure, validation monitoring, and testing, PyTorch provides the best balance of flexibility, transparency, Python integration, and suitability for training the custom CNN used in MyHealer.

In conclusion, PyTorch is selected as the AI training framework for MyHealer because it supports from-scratch CNN development, provides clear control over the training process, integrates well with Python-based services, and allows the model development process to be explained in detail. TensorFlow and Keras are strong alternatives, but they are less suitable for the selected implementation because MyHealer prioritises transparency, custom model control, and smooth integration with the FastAPI prediction service.

2.1.8.7 AI Model Serving Framework

After the CNN model is trained, it must be made accessible to the MyHealer hospital management system. This requires an AI model serving framework. The serving framework is responsible for loading the trained model, receiving an uploaded X-ray image, applying preprocessing, running inference, generating the predicted class, calculating the confidence score, producing the Grad-CAM heatmap, and returning the result to the doctor dashboard. Without a serving framework, the trained model would remain only as a local training output and could not be integrated into the actual hospital workflow.

The model serving framework must satisfy several requirements. First, it must be Python-based because the model is trained using PyTorch and the Grad-CAM implementation will also be handled in Python. Second, it must support image upload and structured API responses. Third, it must be lightweight enough for a solo FYP developer to implement and test. Fourth, it must separate the AI prediction process from the main Django hospital management back end, so that the AI module can be maintained, replaced, or improved without heavily modifying the core hospital system. Lastly, it must be suitable for a prototype system where development speed, clarity, and integration feasibility are important.

Four AI model serving options were considered for MyHealer: FastAPI, Flask, Django REST Framework, and TorchServe. FastAPI is a modern Python framework for building APIs using standard Python type hints and provides automatic interactive documentation (FastAPI, n.d.). Flask is a lightweight WSGI web application framework designed to make development quick and easy while allowing applications to scale to more complex use cases (Pallets, n.d.). Django REST Framework is a toolkit for building Web APIs on top of Django (Encode OSS Ltd., n.d.-a). TorchServe is a tool for serving PyTorch models, but its official documentation states that it is no longer actively maintained and that no planned updates, bug fixes, new features, or security patches are expected (PyTorch, n.d.-c). Table 2.13 summarises the comparison of these serving options.

Table 2.13: A Summarized Comparison of AI Model Serving Frameworks

Aspect	FastAPI (FastAPI, n.d.)	Flask (Pallets, n.d.)	Django REST Framework	TorchServe (PyTorch,
--------	-------------------------	-----------------------	-----------------------	----------------------

			(Encode OSS Ltd., n.d.-a)	n.d.-c)
Main concept	Modern Python API framework based on standard Python type hints	Lightweight Python web framework	REST API toolkit for Django applications	Model serving tool designed for PyTorch models
Main role in MyHealer	Separate AI prediction API	Possible lightweight AI API	Main hospital management API, possible but less ideal for AI serving	Possible PyTorch model serving option
Strengths	Fast API development; automatic interactive documentation; type validation; suitable for Python machine learning services	Simple and flexible; easy to start; suitable for small APIs	Already used in the main system; strong integration with Django models, authentication, and permissions	Designed specifically for serving PyTorch models
Weaknesses	Requires a separate service from Django; service communication must be handled	More manual setup for validation and documentation; less structured than FastAPI for	Coupling AI inference directly into the hospital back end may reduce separation	No longer actively maintained; future updates, fixes, and security

	carefully	typed APIs	of concerns	patches are not expected
Suitability for MyHealer	High	Medium	Medium	Low

Based on Table 2.13, FastAPI is selected as the most suitable AI model serving framework for MyHealer. The main reason is that it provides a clean and lightweight way to expose the trained PyTorch model through an API. A FastAPI endpoint can be designed to accept an uploaded X-ray image, preprocess the image, run the CNN model, generate the fractured or non-fractured prediction, calculate the confidence score, create the Grad-CAM heatmap, and return the output in a structured response. This supports the doctor workflow because the React dashboard can request an AI prediction and display the returned result to the doctor.

FastAPI is also suitable because it provides automatic interactive API documentation (FastAPI, n.d.). This is useful during development because the prediction endpoint can be tested directly through the generated documentation interface. For a solo developer, this reduces the time needed to manually test requests and responses. It also helps verify whether the API accepts images correctly and returns the expected prediction fields. In MyHealer, this is useful because the AI service must communicate correctly with the main hospital management system and the doctor dashboard.

Another reason FastAPI is selected is that it supports separation of concerns. Django with Django REST Framework remains responsible for hospital management functions, including users, roles, appointments, medical records, prescriptions, pharmacy, billing, queue records, and hospitalisation. FastAPI is responsible only for AI prediction. This separation is beneficial because AI inference may require separate dependencies, model files, image processing libraries, Grad-CAM implementation, and possibly GPU-related configuration. Keeping the AI service separate reduces the risk of making the main Django back end too complex.

Flask was considered because it is lightweight and easy to start. It can also be used to build a

simple prediction API. However, FastAPI provides stronger built-in support for Python type hints, request validation, and automatic documentation. These features make FastAPI more suitable for a structured AI prediction service where the input and output format should be clear. Flask remains a possible alternative, but it would require more manual setup to achieve the same level of validation and documentation support.

Django REST Framework was also considered because it is already selected for the main MyHealer back end. Using Django REST Framework to serve the AI model directly would reduce the number of separate services. However, this would tightly couple AI inference with the hospital management back end. Since the AI module involves model loading, image preprocessing, prediction, confidence scoring, and Grad-CAM generation, it is more maintainable to keep it as a separate FastAPI service. Django REST Framework should remain responsible for storing and managing hospital data, while FastAPI should handle the AI prediction process.

TorchServe was considered because it is designed specifically for serving PyTorch models. However, its official documentation states that the project is no longer actively maintained, and that existing releases remain available without planned updates, bug fixes, new features, or security patches (PyTorch, n.d.-c). This makes TorchServe unsuitable for a new healthcare-related FYP prototype because long-term maintainability and security are important concerns. Even though MyHealer is an academic prototype, selecting a tool with limited maintenance would be difficult to justify when FastAPI provides a simpler and more actively usable alternative.

The main weakness of selecting FastAPI is that it introduces an additional service into the system. MyHealer must run both the Django back end and the FastAPI AI service. This means that service communication, error handling, image transfer, endpoint security, deployment configuration, and service availability must be managed carefully. For example, if the FastAPI service is offline, the doctor dashboard may not be able to obtain AI predictions even though the hospital management system remains available. This weakness can be reduced by designing clear error messages, storing uploaded X-ray records in Django, and allowing the system to show prediction status when the AI service is unavailable.

In conclusion, FastAPI is selected as the AI model serving framework for MyHealer because it is lightweight, Python-based, suitable for machine learning APIs, and supports automatic

API documentation. It allows the trained PyTorch CNN model to be exposed as a prediction service that can be called by the doctor dashboard. Flask and Django REST Framework are possible alternatives, but FastAPI provides a better balance of simplicity, structure, and separation of concerns. TorchServe is not selected because its limited maintenance status makes it unsuitable for a new system. Together, PyTorch and FastAPI form a clear AI pipeline for MyHealer: PyTorch is used to train and evaluate the CNN model, while FastAPI is used to serve the trained model as an AI prediction API.

2.2 Feasibility Study

The feasibility study evaluates whether MyHealer can be realistically developed, implemented, and maintained within the scope of this final year project. Since MyHealer combines an integrated hospital management platform with an AI-assisted bone fracture classification module, feasibility must be assessed from several perspectives: technical, economic, operational, schedule, legal, and ethical. This is necessary because the project does not only involve ordinary hospital information management functions, but also handles medical images, patient-related records, AI-generated diagnostic support, and role-based access for different hospital users.

Overall, the project is considered feasible as an academic prototype. The hospital management components can be developed using established web and mobile technologies, while the AI-assisted fracture classification module can be developed using a supervised learning approach with a publicly available X-ray dataset. However, the system must be clearly positioned as a prototype and diagnostic support tool, not as a clinically approved replacement for doctors or radiologists. The following subsections discuss the feasibility of MyHealer in greater detail.

2.2.1 Technical Feasibility

MyHealer is technically feasible because the technologies selected for the project are mature, widely documented, and suitable for the system architecture. The patient-facing mobile application is developed using Flutter, while the doctor, staff, pharmacist, and administrator dashboards are developed using React. The main back end is developed using Django with Django REST Framework, and PostgreSQL is selected as the database. The AI module is trained using PyTorch and served through FastAPI. This technology stack is technically coherent because the back-end and AI components are both Python-based, reducing integration complexity between the hospital management platform and the fracture classification service.

The technical feasibility of the patient application is supported by Flutter's cross-platform capability. Since patients may use either Android or iOS devices, developing two separate native applications would increase development time and maintenance effort. Flutter allows a single codebase to target both major mobile platforms, which is suitable for a solo developer and supports the mobile requirements of MyHealer, such as appointment booking, queue monitoring, record viewing, and notifications (Google, n.d.-a). React is also technically suitable for the web dashboards because MyHealer requires reusable interface components, dynamic pages, dashboard layouts, tables, forms, and API-driven data updates (Meta, n.d.-a).

The back-end and database layers are also feasible. Django provides a structured framework for managing relational workflows such as patient registration, appointments, prescriptions, billing, ward admission, and staff administration (Django Software Foundation, n.d.-a). Django REST Framework provides the API layer required by the Flutter and React clients (Encode OSS Ltd., n.d.-a). PostgreSQL is suitable because the system contains highly related data entities, including patients, doctors, appointments, prescriptions, medical records, payments, wards, beds, and AI prediction results. Its relational structure, transaction support, and compatibility with Django make it appropriate for storing structured hospital data (PostgreSQL Global Development Group, n.d.-a; Django Software Foundation, n.d.-f).

The AI module is also technically feasible because the selected task is limited to binary classification. The model classifies an X-ray image as fractured or non-fractured. It does not attempt to classify fracture types, generate bounding boxes, or perform segmentation. This

controlled scope reduces technical complexity and matches the selected Bone Fracture Multi-Region X-ray Data dataset, which is organised into fractured and non-fractured folders (bmadushanirodrigo, n.d.). PyTorch supports the development of a custom CNN from scratch, while FastAPI provides a suitable interface for serving the trained model as an AI prediction API (PyTorch, n.d.-a; FastAPI, n.d.).

However, several technical risks must be acknowledged. First, training a CNN requires careful preprocessing, validation, and tuning. Poor preprocessing or overfitting may reduce model performance. Second, Grad-CAM provides only a visual explanation of the model's attention, not an exact fracture boundary (Selvaraju et al., 2017). Third, integration between Django and FastAPI must be designed carefully so that image uploads, prediction requests, error handling, and returned results are handled consistently. Despite these risks, the project remains technically feasible because the AI scope is controlled, the dataset is suitable for classification, and the technologies selected are appropriate for the system requirements.

2.2.2 Economic Feasibility

Economic feasibility evaluates whether MyHealer can be developed, operated, and maintained within a reasonable cost. Since MyHealer is developed as an academic prototype, the project does not require commercial software licences because the selected technologies, including Flutter, React, Django, Django REST Framework, PostgreSQL, PyTorch, and FastAPI, are free and open-source. Therefore, the main economic concern is not software licensing, but the estimated cost of deploying, supporting, maintaining, and improving the system after development.

For the purpose of this feasibility study, the cost estimation is based on a small-scale prototype deployment. The estimation assumes that MyHealer will be hosted using affordable cloud or server resources, with additional costs for backup storage, system maintenance, support, and occasional AI model updates. The estimation does not include the student developer's personal laptop or development computer, because these are existing development resources rather than direct project costs.

2.2.2.1 Cost and Benefit Summary

Table 2.14 summarises the estimated cost of developing, operating, and maintaining MyHealer over five years.

Table 2.14: Summary of Estimated Cost for MyHealer

Cost Category	Year 0 (RM)	Year 1 (RM)	Year 2 (RM)	Year 3 (RM)	Year 4 (RM)	Year 5 (RM)	Total Cost (RM)
Initial Development and Deployment Cost	2,500	-	-	-	-	-	2,500
Hosting and Operating Cost	-	1,800	2,100	2,400	2,700	3,000	12,000
System Support and Maintenance Cost	-	1,500	2,000	2,000	2,500	2,500	10,500
AI Model Update and Testing Cost	-	-	1,200	-	1,200	-	2,400
Total Cost	2,500	3,300	5,300	4,400	6,400	5,500	27,400

Based on Table 2.14, the estimated five-year cost of MyHealer is RM27,400. The highest recurring expenses are hosting, operation, support, and maintenance. These costs are reasonable because MyHealer relies on open-source software and does not require paid software licences. The estimated cost may increase if the system is deployed in a real hospital environment with larger storage, stronger security controls, production-grade cloud infrastructure, and external clinical validation. However, for an academic prototype and small-scale demonstration, the estimated cost remains manageable.

Table 2.15 summarises the estimated tangible benefits that may be obtained from MyHealer over five years.

Table 2.15: Summary of Estimated Benefit for MyHealer

Benefit Category	Year 0 (RM)	Year 1 (RM)	Year 2 (RM)	Year 3 (RM)	Year 4 (RM)	Year 5 (RM)	Total Benefit (RM)
Administrative Time Savings	-	4,800	5,280	5,808	6,389	7,028	29,305
Queue and Registration Efficiency Gained	-	3,600	3,960	4,356	4,792	5,271	21,979
Reduction in Record and Processing Errors	-	1,200	1,320	1,452	1,597	1,757	7,326
Diagnostic Review Support Efficiency	-	4,000	4,400	4,840	5,324	5,856	24,420
Total Benefit	-	13,600	14,960	16,456	18,102	19,912	83,030

Based on Table 2.15, the estimated five-year benefit is RM83,030. These benefits are mainly obtained through reduced manual administrative work, improved queue and registration efficiency, fewer record-processing errors, and time savings during diagnostic review. Since MyHealer integrates appointment handling, queue management, medical records, prescription workflows, and AI-assisted fracture classification into one platform, the system can reduce duplicated work and improve operational efficiency.

The estimated net benefit over five years is calculated as follows:

Net Benefit = Total Estimated Benefit - Total Estimated Cost

Net Benefit = RM83,030 - RM27,400

Net Benefit = RM55,630

This shows that MyHealer is economically feasible under the assumed cost and benefit estimation. Although the figures are estimated and may vary in real deployment, the result suggests that the system can provide more value than its operating and maintenance cost over time.

2.2.2.2 Tangible Cost

Tangible costs refer to costs that can be directly measured in monetary value. For MyHealer, tangible costs mainly include initial deployment setup, hosting, storage, system support, maintenance, and AI model update activities. Table 2.16 summarises the estimated one-time development and deployment cost.

Table 2.16: Summary of One-Time Development and Deployment Cost

One-Time Cost Item	Estimated Cost (RM)	Quantity	Total Cost (RM)
Initial cloud/server configuration	1,000	1	1,000
Domain, SSL, and deployment setup	300	1	300
Database and backup configuration	500	1	500
AI model training and testing resources	400	1	400
Documentation and user testing materials	200	1	200
Contingency allowance	100	1	100
Total Cost			2,500

The one-time development and deployment cost is estimated at RM2,500. This cost is mainly allocated to preparing the system for demonstration and small-scale deployment. The initial cloud or server configuration is needed to host the web back end, database, and AI prediction service. The domain, SSL, and deployment setup cost supports secure access to the system. Database and backup configuration is included because MyHealer handles patient records, appointments, prescriptions, AI prediction results, and hospitalisation records, all of which require reliable storage. The AI model training and testing resource cost is included to cover possible computing resources used during model experimentation and evaluation.

This cost is considered reasonable because MyHealer does not require paid software licences. The system is developed using free and open-source tools, so the financial burden is mainly

related to deployment and operation rather than software purchase. Since the project is an academic prototype, the cost can also be reduced further by using local development environments and free-tier cloud services during early testing.

2.2.2.3 Operating and Maintenance Cost

Operating and maintenance costs refer to the recurring expenses required to keep the system available, updated, and reliable. For MyHealer, these include hosting, backup storage, technical support, system maintenance, and periodic AI model updates. Table 2.17 summarises the estimated operating and maintenance costs over five years.

Table 2.17: Summary of Operating and Maintenance Cost

Cost Item	Year 1 (RM)	Year 2 (RM)	Year 3 (RM)	Year 4 (RM)	Year 5 (RM)	Total Cost (RM)
Hosting and server operation	1,200	1,400	1,600	1,800	2,000	8,000
Backup storage and file management	600	700	800	900	1,000	4,000
System support	800	1,000	1,000	1,200	1,200	5,200
System maintenance	700	1,000	1,000	1,300	1,300	5,300
AI model update and testing	-	1,200	-	1,200	-	2,400
Total Cost	3,300	5,300	4,400	6,400	5,500	24,900

The total estimated operating and maintenance cost over five years is RM24,900. Hosting and server operation costs are expected to increase gradually because the system may store more patient records, medical images, prescription records, and AI prediction results over time. Backup storage and file management costs are also expected to increase because X-ray images and Grad-CAM outputs may require additional storage capacity.

System support is included because users may require assistance when using the platform, especially during the early stage of adoption. System maintenance is also necessary to fix bugs, update dependencies, improve security, and refine system functions based on feedback.

AI model update and testing cost is estimated in Year 2 and Year 4 because the model may need to be retrained, re-evaluated, or tested with improved preprocessing and additional validation data. This cost is not included every year because the AI model does not need to be retrained continuously for the prototype.

2.2.2.4 Intangible Cost

Intangible costs refer to non-monetary costs that may affect the adoption and effectiveness of MyHealer. One major intangible cost is the learning curve for users. Patients, doctors, hospital staff, pharmacists, and administrators may need time to understand the new system. Doctors may need to learn how to upload X-ray images, interpret AI prediction results, review confidence scores, and inspect Grad-CAM heatmaps. Hospital staff may need to adapt to digital queue handling, appointment management, and radiology image workflows. Pharmacists and administrators may also need time to become familiar with their respective system modules.

Another intangible cost is resistance to change. Some hospital users may be more comfortable with manual processes or existing systems. If users are not confident in the platform, they may avoid using it fully or may continue relying on manual workarounds. This can reduce the expected efficiency benefits of MyHealer. To reduce this issue, user training, simple interface design, clear system instructions, and gradual adoption are necessary.

There is also an intangible cost related to trust in AI-assisted diagnosis. Doctors may be cautious about relying on a CNN prediction because the model may produce false positives or false negatives. This concern is valid because the system is only an academic prototype and has not been clinically validated. To reduce this risk, MyHealer presents the AI output as decision support rather than a final diagnosis. The confidence score and Grad-CAM heatmap are provided to help doctors review the result critically, while the final medical decision remains under the doctor's responsibility.

2.2.2.5 Economic Feasibility Conclusion

Overall, MyHealer is economically feasible as an academic prototype. The project avoids high software licensing costs because all major technologies used are free and open-source. The estimated five-year cost is RM27,400, while the estimated five-year benefit is RM83,030, producing an estimated net benefit of RM55,630. Although these figures are based on assumptions, they show that the system has a reasonable cost-benefit balance.

The main economic risks are future hosting cost, maintenance effort, AI model update cost, and the possible cost of real clinical validation if the system is expanded beyond the prototype stage. However, these risks are acceptable within the current FYP scope because MyHealer is not intended for immediate hospital deployment. Therefore, the project is considered economically feasible for development, demonstration, and academic evaluation.

2.2.3 Operational Feasibility

MyHealer is operationally feasible because it supports existing hospital workflows rather than replacing them entirely. The system is designed around familiar hospital roles, including patients, doctors, hospital staff, pharmacists, and administrators. Patients use the mobile application for appointment booking, queue monitoring, medical record viewing, and payment. Doctors use the web dashboard for consultation, diagnosis, prescription, and AI-assisted fracture classification. Hospital staff manage patient registration, appointments, queues, and radiology images. Pharmacists manage prescriptions and medication distribution, while administrators manage users, reports, and hospital operations.

The AI-assisted fracture classification module is operationally suitable because it is positioned as decision support. The system provides a fractured or non-fractured prediction, confidence score, and Grad-CAM heatmap, but the final diagnosis remains under the doctor's responsibility. This approach is more acceptable than fully automated diagnosis because it preserves professional medical judgement. Existing studies support the value of AI assistance in fracture-related workflows, where AI can improve clinician performance when used as a supporting tool rather than a replacement (Lindsey et al., 2018; Duron et al., 2021).

The system may also improve operational efficiency by centralising hospital functions. Instead of separating appointment handling, queue monitoring, medical records, prescription processing, and radiology review into disconnected processes, MyHealer integrates them into one platform. This can reduce duplicated work and improve access to information during consultation. The AI module can further support doctors by drawing attention to fracture-related patterns in X-ray images, while the Grad-CAM heatmap provides a visual explanation that can be reviewed before making a decision.

However, operational challenges may still occur. Hospital users may require training before they can use the system effectively. Doctors may also be cautious about AI-generated results, especially if the model occasionally produces false positives or false negatives. In addition, the system depends on proper data entry and correct image upload. If staff upload the wrong X-ray image, enter incorrect patient information, or ignore system alerts, the operational value of the system may be reduced. These risks can be reduced through clear user interface design, role-based access control, validation rules, confirmation steps, user training, and audit

logs.

Overall, MyHealer is operationally feasible as a prototype because its functions align with hospital workflows and its AI module is designed to support rather than replace doctors. The system may require training and workflow adjustment, but its role-based structure and integrated design make it suitable for demonstration in an academic hospital management context.

2.2.4 Schedule Feasibility

MyHealer is schedule feasible if the project is developed according to the planned scope and prioritised carefully. The system contains many modules, including patient management, appointment and queue management, consultation, AI fracture classification, prescription, pharmacy, administration, and hospitalisation. Since the project is developed by a solo developer, the schedule must focus first on the core functions and avoid unnecessary expansion.

The most important schedule priority is the AI-assisted fracture classification module because it is the central contribution of the project. The CNN model, preprocessing pipeline, training process, evaluation results, Grad-CAM output, and FastAPI prediction endpoint should be developed before non-essential enhancements. The second priority is the minimum hospital workflow needed to demonstrate AI integration, including doctor login, patient record access, X-ray image upload, prediction result display, and saving of AI results into the consultation record. Supporting modules such as pharmacy, payment, queue monitoring, and hospitalisation can then be developed based on the remaining time.

Scrum is suitable for managing the schedule because the project can be divided into short development cycles. Each sprint can focus on a specific group of features, such as user authentication, patient appointment workflow, doctor consultation workflow, AI model training, AI service integration, pharmacy functions, and report generation. This allows progress to be reviewed regularly and helps identify delays earlier. Since the project is developed individually, Scrum must be adapted to a solo development context, where the student acts as developer, tester, and project manager.

The main schedule risk is scope overload. MyHealer is a large system because it includes both a hospital management platform and an AI model. If all modules are developed with full production-level detail, the project may exceed the available timeline. To reduce this risk, the prototype should prioritise completeness of the core AI workflow over excessive detail in supporting modules. For example, the pharmacy and hospitalisation modules can be implemented with essential create, view, update, and status-management functions, while the AI fracture classification workflow should receive the strongest implementation and evaluation effort.

Overall, the project is schedule feasible if development is managed incrementally and the AI module remains the main priority. The schedule should avoid uncontrolled feature expansion and should treat supporting hospital management functions as the delivery platform for the AI diagnostic support module.

2.2.5 Legal Feasibility

MyHealer is legally feasible as an academic prototype, provided that it is not presented as a clinically approved diagnostic tool and does not process real patient data without proper authorisation. The system handles sensitive information such as patient profiles, medical records, X-ray images, prescriptions, payments, and hospitalisation records. Therefore, legal feasibility depends mainly on data protection, consent, system access control, and the regulatory status of AI-assisted diagnostic software.

In Malaysia, personal data protection is governed by the Personal Data Protection Act 2010. The Act regulates the processing of personal data in commercial transactions and protects the interests of data subjects (Department of Personal Data Protection Malaysia, n.d.). Since MyHealer stores and processes patient-related data, the system design must follow data protection principles such as limiting access to authorised users, collecting only necessary information, protecting stored records, and avoiding unauthorised disclosure. In the prototype, real patient data should not be used unless proper permission, anonymisation, and ethical approval are obtained.

Medical images also require careful handling. X-ray images may contain patient identifiers or be linked to medical records. Therefore, uploaded images should be stored securely, linked only to the correct patient record, and accessible only to authorised doctors or staff. If sample images from public datasets are used for development and demonstration, they should be used according to the dataset's permitted usage conditions. The system should also avoid displaying or exporting sensitive information unnecessarily.

Legal feasibility also depends on the regulatory status of the AI module. The Medical Device Authority Malaysia is responsible for regulating medical devices and enforcing medical device law under the Medical Device Act 2012 (Medical Device Authority, n.d.). Since MyHealer's AI module provides diagnostic support for fracture classification, real clinical deployment may require assessment under the relevant medical device regulatory framework. However, within the current FYP scope, MyHealer is only an academic prototype and is not intended for clinical diagnosis, commercial distribution, or hospital deployment. Therefore, the report should clearly state that the AI output is for decision-support demonstration only and must not be used as a final medical diagnosis.

The main legal risks are unauthorised use of real patient data, misleading claims about diagnostic accuracy, and deployment without regulatory clearance. These risks can be reduced by using public datasets, anonymised test data, clear disclaimers, role-based access control, secure authentication, and careful wording in the report. MyHealer should avoid claiming that it is clinically validated or approved. Instead, it should be described as an academic prototype that demonstrates how AI-assisted fracture classification can be integrated into hospital workflows.

2.2.6 Ethical Feasibility

MyHealer is ethically feasible if the system is designed and presented as a supportive tool that improves medical workflow without replacing human responsibility. The most important ethical concern is patient safety. A false negative result, where a fractured X-ray is classified as non-fractured, may delay treatment if the doctor relies on the AI result without review. A false positive result may cause unnecessary concern or further examination. Therefore, the AI module must not be treated as an autonomous diagnostic authority. The doctor must remain responsible for reviewing the X-ray image, considering the patient's symptoms, and making the final diagnosis.

Transparency is another ethical concern. Deep learning models can be difficult to interpret because their decision-making process is not always visible. MyHealer addresses this concern by using Grad-CAM to generate a heatmap that shows the image regions influencing the model's prediction (Selvaraju et al., 2017). This improves transparency by allowing doctors to inspect whether the model appears to focus on relevant bone regions. However, Grad-CAM does not provide exact fracture localisation and should not be interpreted as a confirmed medical explanation. It is only a visual aid for understanding model attention.

Bias and generalisation must also be considered. The CNN is trained using a public X-ray dataset, which may not fully represent local hospital patients, imaging machines, clinical protocols, or demographic variation. As a result, the model may perform differently when applied to real hospital images. Deep learning systems in medical imaging require careful validation before clinical use, and performance reported on one dataset may not guarantee safe performance in another setting (Aggarwal et al., 2021). Therefore, MyHealer's AI module should be evaluated as a prototype and not as a clinically validated diagnostic system.

Privacy is also an ethical requirement. Patient records and X-ray images should only be accessed by authorised users. The system should apply role-based access control so that patients, doctors, staff, pharmacists, and administrators only access the information needed for their duties. Sensitive data should not be exposed unnecessarily, and test data should be anonymised where possible. These practices support both ethical responsibility and legal data protection requirements.

Another ethical issue is over-reliance on AI. If doctors or staff trust the AI output too strongly, they may overlook clinical evidence or image features that contradict the model's prediction. To reduce this risk, MyHealer should display the AI result as "AI-assisted prediction" rather than "final diagnosis." The interface should also show the confidence score and Grad-CAM heatmap together, encouraging doctors to review the result critically. The system should not automatically approve treatment decisions based only on the CNN output.

In conclusion, MyHealer is ethically feasible because it maintains human oversight, provides interpretability through Grad-CAM, uses a controlled prototype scope, and recognises the limitations of public-dataset training. Its ethical acceptability depends on careful presentation: the AI module must support doctors, not replace them, and the system must avoid misleading claims about clinical readiness. Future real-world deployment would require stronger validation, ethical approval, regulatory review, and testing using representative clinical data.

2.3 Chapter Summary and Evaluation

This chapter reviewed the literature, existing systems, technologies, datasets, and feasibility factors relevant to the development of MyHealer. The review established that MyHealer is primarily an AI-assisted bone fracture classification system embedded within an integrated hospital management platform. The hospital management functions, including appointment handling, queue monitoring, consultation, prescription, pharmacy, administration, and hospitalisation, serve as the supporting environment for the AI module rather than the main research contribution. The central contribution remains the use of a convolutional neural network to classify X-ray images as fractured or non-fractured, supported by a confidence score and Grad-CAM visual explanation.

The comparison of existing systems showed that current healthcare and hospital applications mainly focus on administrative or patient-facing functions. Systems such as MySejahtera, KPJ Cares, and Epic MyChart support healthcare access, appointments, records, or enterprise hospital workflows, but they do not provide AI-assisted fracture classification within the patient and doctor workflow. On the other hand, existing commercial AI fracture detection tools provide diagnostic support but are generally proprietary, enterprise-oriented, and not embedded within an accessible hospital management platform. Therefore, MyHealer addresses a gap by combining hospital workflow integration with AI-assisted fracture classification.

The technology review justified the selection of the main implementation stack. Flutter was selected for the mobile application because it supports cross-platform development from a single codebase, making it suitable for a patient-facing application developed by a solo developer. React was selected for the web front end because it supports reusable components and dashboard-oriented development, which are needed for doctor, staff, pharmacist, and administrator interfaces. Django with Django REST Framework was selected as the main back-end framework because it provides structured data handling, authentication, permissions, administrative support, and REST API development. PostgreSQL was selected as the database because MyHealer requires reliable relational data storage for interconnected hospital records. REST was selected as the API architectural style because it fits the resource-based workflows of the system and integrates well with Django REST Framework.

The deep learning review further justified the AI approach used in MyHealer. CNNs were

found to be suitable for X-ray image classification because they can learn hierarchical image features directly from training data, reducing dependency on manually designed image-processing rules. A custom CNN trained from scratch was selected because it aligns with the binary classification scope and allows the model architecture and training process to be explained clearly in the final year project report. Existing studies on fracture detection and medical imaging support the feasibility of AI-assisted radiograph analysis, while also showing that AI should be used as a support tool rather than a replacement for clinical judgement.

The dataset review concluded that Bone Fracture Multi-Region X-ray Data is suitable for MyHealer because it provides fractured and non-fractured image folders organised into training, validation, and testing sets. This structure matches the selected binary classification task. Since the dataset provides image-level labels rather than bounding box or segmentation annotations, MyHealer does not perform object detection or exact fracture localisation. Instead, Grad-CAM is used to generate a heatmap showing the image regions that influenced the model's classification result. This allows the doctor to inspect the AI output visually while still maintaining responsibility for the final diagnosis.

The feasibility study showed that MyHealer is feasible as an academic prototype. Technically, the selected tools are mature, well documented, and compatible with the system architecture. Economically, the project is feasible because it relies mainly on free and open-source technologies. Operationally, the system fits existing hospital roles and supports workflow integration without replacing medical professionals. The project is schedule feasible if development prioritises the AI classification workflow and essential hospital functions before optional enhancements. Legally and ethically, MyHealer is feasible only if it is clearly positioned as a prototype and decision-support tool, not as a clinically approved diagnostic system. Patient privacy, data protection, role-based access control, human oversight, and avoidance of misleading claims are essential considerations.

Overall, the evaluation of the literature and technologies supports the development of MyHealer. The selected architecture provides a practical foundation for building an integrated hospital management platform, while the AI module provides the project's main innovation through CNN-based fracture classification and Grad-CAM interpretability. The review also confirms that the project scope is appropriate: a binary fractured or non-fractured classification model is technically achievable, fits the selected dataset, and can be integrated

into the doctor workflow as decision support. The findings from this chapter therefore provide the basis for the methodology, system requirements, design, implementation, and testing activities discussed in the following chapters.

References

- Abdeen, I., Rahman, M. A., Prottyasha, F. Z., Ahmed, T., Chowdhury, T. M., & Shatabda, S. (2023). FracAtlas: A dataset for fracture classification, localization and segmentation of musculoskeletal radiographs. *Scientific Data*, 10, Article 521. <https://doi.org/10.1038/s41597-023-02432-4>
- Aggarwal, R., Sounderajah, V., Martin, G., Ting, D. S. W., Karthikesalingam, A., King, D., Ashrafian, H., & Darzi, A. (2021). Diagnostic accuracy of deep learning in medical imaging: A systematic review and meta-analysis. *npj Digital Medicine*, 4, Article 65. <https://doi.org/10.1038/s41746-021-00438-z>
- Alam, A., Al-Shamayleh, A. S., Thalji, N., Raza, A., Morales Barajas, E. A., Bautista Thompson, E., de la Torre Diez, I., & Ashraf, I. (2025). Novel transfer learning based bone fracture detection using radiographic images. *BMC Medical Imaging*, 25, Article 5. <https://doi.org/10.1186/s12880-024-01546-4>
- Applied Radiology. (2022, March 4). *AI software to detect and localize fractures gets FDA nod*. <https://appliedradiology.com/articles/ai-software-to-detect-and-localize-fractures-gets-fda-nod>
- bmadushanirodrigo. (n.d.). *Bone Fracture Multi-Region X-ray Data* [Data set]. Kaggle. <https://www.kaggle.com/datasets/bmadushanirodrigo/fracture-multi-region-x-ray-data>
- Biørn-Hansen, A., Grønli, T.-M., & Ghinea, G. (2018). A survey and taxonomy of core concepts and research challenges in cross-platform mobile development. *ACM Computing Surveys*, 51(5), Article 108. <https://doi.org/10.1145/3241739>
- Biørn-Hansen, A., Grønli, T.-M., Ghinea, G., & Alouneh, S. (2019). An empirical study of cross-platform mobile development in industry. *Wireless Communications and Mobile Computing*, 2019, Article 5743892. <https://doi.org/10.1155/2019/5743892>
- Boo, S.-L. (2025, July 7). *Public sector short of nearly 11,000 specialist doctors: MOH data*. CodeBlue. <https://codeblue.galencentre.org/2025/07/public-sector-short-of-nearly-11000-specialist-doctors-moh-data/>
-

Django Software Foundation. (n.d.-a). *Django: The web framework for perfectionists with deadlines*. Retrieved July 2026, from <https://www.djangoproject.com/>

Django Software Foundation. (n.d.-b). *Models*. Django documentation. Retrieved July 2026, from <https://docs.djangoproject.com/en/6.0/topics/db/models/>

Django Software Foundation. (n.d.-c). *Using the Django authentication system*. Django documentation. Retrieved July 2026, from <https://docs.djangoproject.com/en/6.0/topics/auth/default/>

Django Software Foundation. (n.d.-d). *The Django admin site*. Django documentation. Retrieved July 2026, from <https://docs.djangoproject.com/en/6.0/ref/contrib/admin/>

Django Software Foundation. (n.d.-e). *Security in Django*. Django documentation. Retrieved July 2026, from <https://docs.djangoproject.com/en/6.0/topics/security/>

Django Software Foundation. (n.d.-f). *Databases*. Django documentation. Retrieved July 2026, from <https://docs.djangoproject.com/en/6.0/ref/databases/>

Django Software Foundation. (n.d.-g). *django.contrib.postgres*. Django documentation. Retrieved July 2026, from <https://docs.djangoproject.com/en/6.0/ref/contrib/postgres/>

Duron, L., Ducarouge, A., Gillibert, A., Lainé, J., Allouche, C., Cherel, N., Zhang, Z., Niche, N., Lacave, E., Pourchot, A., Felter, A., Lassalle, L., Regnard, N. E., & Feydy, A. (2021). Assessment of an AI aid in detection of adult appendicular skeletal fractures by emergency physicians and radiologists: A multicenter cross-sectional diagnostic study. *Radiology*, 300(1), 120-129. <https://doi.org/10.1148/radiol.2021203886>

Encode OSS Ltd. (n.d.-a). *Django REST framework*. Retrieved July 2026, from <https://www.django-rest-framework.org/>

Encode OSS Ltd. (n.d.-b). *ViewSet*. Django REST framework. Retrieved July 2026, from <https://www.django-rest-framework.org/api-guide/viewsets/>

Epic Systems Corporation. (n.d.). *MyChart*. Retrieved July 2026, from <https://www.mychart.org/>

FastAPI. (n.d.). *FastAPI*. Retrieved July 2026, from <https://fastapi.tiangolo.com/>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://roy.gbiv.com/pubs/dissertation/rest_arch_style.htm

Google. (n.d.-a). *Flutter documentation*. Retrieved July 2026, from <https://docs.flutter.dev/>

Google. (n.d.-b). *What is Angular?* Angular. Retrieved July 2026, from <https://angular.dev/overview>

GraphQL Foundation. (n.d.). *GraphQL: The query language for modern APIs*. Retrieved July 2026, from <https://graphql.org/>

gRPC Authors. (n.d.). *Introduction to gRPC*. Retrieved July 2026, from <https://grpc.io/docs/what-is-grpc/introduction/>

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770-778). <https://doi.org/10.1109/CVPR.2016.90>

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4700-4708). <https://doi.org/10.1109/CVPR.2017.243>

Keras. (n.d.). *Keras: Deep learning for humans*. Retrieved July 2026, from <https://keras.io/>

KPJ Healthcare Berhad. (n.d.). *KPJ Cares app*. Retrieved July 2026, from <https://www.kpjhealth.com.my/cares-app/faq>

Laravel. (n.d.-a). *Authentication*. Laravel 13.x documentation. Retrieved July 2026, from <https://laravel.com/docs/13.x/authentication>

Laravel. (n.d.-b). *Validation*. Laravel 13.x documentation. Retrieved July 2026, from <https://laravel.com/docs/13.x/validation>

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436-444. <https://doi.org/10.1038/nature14539>

Lindsey, R., Daluiski, A., Chopra, S., Lachapelle, A., Mozer, M., Sicular, S., Hanel, D., Gardner, M., Gupta, A., Hotchkiss, R., & Potter, H. (2018). Deep neural network improves

fracture detection by clinicians. *Proceedings of the National Academy of Sciences*, 115(45), 11591-11596. <https://doi.org/10.1073/pnas.1806905115>

Meta. (n.d.-a). *React: The library for web and native user interfaces*. Retrieved July 2026, from <https://react.dev/>

Meta. (n.d.-b). *React Native documentation*. Retrieved July 2026, from <https://reactnative.dev/>

Ministry of Health Malaysia. (n.d.). *MySejahtera*. Retrieved July 2026, from <https://mysejahtera.moh.gov.my/en/>

MongoDB. (n.d.). *What is MongoDB? MongoDB documentation*. Retrieved July 2026, from <https://www.mongodb.com/docs/manual/>

MUI. (n.d.). *Material UI: Ready-to-use Material Design components*. Retrieved July 2026, from <https://mui.com/material-ui/>

Nagy, E., Janisch, M., Hrzić, F., Sorantin, E., & Tschauer, S. (2022). A pediatric wrist trauma X-ray dataset (GRAZPEDWRI-DX) for machine learning. *Scientific Data*, 9, Article 222. <https://doi.org/10.1038/s41597-022-01328-z>

Naim, N., Kuroda, Y., Yamada, S., Mori, Y., Liu, C., Espinoza, R., Yamamoto, G., & Kuroda, T. (2025). The evolution of healthcare digitalisation policies in Malaysia: A four-decade narrative review (1985-2025). *Digital Health*, 11. <https://doi.org/10.1177/20552076251353279>

Olczak, J., Fahlberg, N., Maki, A., Razavian, A. S., Jilert, A., Stark, A., Sköldenberg, O., & Gordon, M. (2017). Artificial intelligence for analyzing orthopedic trauma radiographs. *Acta Orthopaedica*, 88(6), 581-586. <https://doi.org/10.1080/17453674.2017.1344459>

OpenJS Foundation. (n.d.). *Express: Node.js web application framework*. Retrieved July 2026, from <https://expressjs.com/>

Oracle. (n.d.). *MySQL Reference Manual*. Retrieved July 2026, from <https://dev.mysql.com/doc/>

Pallets. (n.d.). *Flask documentation*. Retrieved July 2026, from

<https://flask.palletsprojects.com/>

PostgreSQL Global Development Group. (n.d.-a). *PostgreSQL: The world's most advanced open source database*. Retrieved July 2026, from <https://www.postgresql.org/>

PostgreSQL Global Development Group. (n.d.-b). *About PostgreSQL*. Retrieved July 2026, from <https://www.postgresql.org/about/>

PostgreSQL Global Development Group. (n.d.-c). *JSON types*. PostgreSQL documentation. Retrieved July 2026, from <https://www.postgresql.org/docs/current/datatype-json.html>

PyTorch. (n.d.-a). *PyTorch documentation*. Retrieved July 2026, from <https://docs.pytorch.org/docs/stable/index.html>

PyTorch. (n.d.-b). *A gentle introduction to torch.autograd*. PyTorch tutorials. Retrieved July 2026, from https://docs.pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html

PyTorch. (n.d.-c). *TorchServe*. PyTorch documentation. Retrieved July 2026, from <https://docs.pytorch.org/serve/>

Rajpurkar, P., Irvin, J., Bagul, A., Ding, D., Duan, T., Mehta, H., Yang, B., Zhu, K., Laird, D., Ball, R. L., Langlotz, C., Shpanskaya, K., Lungren, M. P., & Ng, A. Y. (2017). MURA: Large dataset for abnormality detection in musculoskeletal radiographs. *arXiv*. <https://arxiv.org/abs/1712.06957>

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 618-626). <https://doi.org/10.1109/ICCV.2017.74>

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations*. <https://arxiv.org/abs/1409.1556>

SQLite. (n.d.). *About SQLite*. Retrieved July 2026, from <https://www.sqlite.org/about.html>

Stack Overflow. (2025). *2025 Stack Overflow Developer Survey: Technology*. Retrieved July 2026, from <https://survey.stackoverflow.co/2025/technology>

Statista. (2025). *Malaysia: Smartphone penetration*. Retrieved July 2026, from <https://www.statista.com/statistics/625418/smartphone-user-penetration-in-malaysia/>

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning* (pp. 6105-6114). <https://proceedings.mlr.press/v97/tan19a.html>

TensorFlow. (n.d.). *Introduction to TensorFlow*. Retrieved July 2026, from <https://www.tensorflow.org/learn>

Vue.js. (n.d.). *Introduction*. Vue.js. Retrieved July 2026, from <https://vuejs.org/guide/introduction>

Wei, C.-J., Tsai, W.-C., Tiu, C.-M., Wu, H.-T., Chiou, H.-J., & Chang, C.-Y. (2006). Systematic analysis of missed extremity fractures in emergency radiology. *Acta Radiologica*, 47(7), 710-717. <https://doi.org/10.1080/02841850600806340>

World Wide Web Consortium. (2007). *SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)*. <https://www.w3.org/TR/soap12-part1/>

Department of Personal Data Protection Malaysia. (n.d.). Personal Data Protection Act 2010 [Act 709]. Retrieved July 2026, from <https://www.pdp.gov.my/ppdpv1/en/akta/pdp-act-2010-en/>

Medical Device Authority. (n.d.). Overview of regulatory medical device. Retrieved July 2026, from <https://portal.mda.gov.my/index.php/industry/overview-of-regulatory-medical-devices/overview-of-regulatory-medical-device>